

Загрузка релиза на сервер

Эта инструкция описывает только перенос файлов и базы. Первичный запуск выполняется по инструкции первоначальной настройки.

Текущий проверенный релиз

- ветка: `dev-v1` ;
- коммит: `df2fd4f3171fdf17644eba5e7ea72afc209aa79e` ;
- локальный проверенный dump: `backups/symbolika-release-20260811-000535.dump` ;
- дата финального регресса: `11.08.2026` .

После `git pull` проверьте версию:

```
git rev-parse HEAD
```

Для точного воспроизведения этого выпуска команда должна вернуть `df2fd4f3171fdf17644eba5e7ea72afc209aa79e` .

1. Что переносится

- код — через Git;
- база — отдельным PostgreSQL custom dump `-Fc` ;
- пользовательские файлы Directus — отдельным архивом каталога `uploads` , если они нужны;
- секреты — вручную в серверный `.env` , никогда не через Git;
- макеты на Яндекс Диске переносить не нужно.

Каталог `database` копировать между Windows и Linux нельзя.

2. Подготовить сервер

Нужен Linux-сервер с Docker Engine, Docker Compose plugin, Git и доступом по SSH.

Рекомендуемый каталог:

```
sudo mkdir -p /opt/symbolika
sudo chown "$USER":"$USER" /opt/symbolika
cd /opt/symbolika
```

3. Настроить доступ GitHub

Предпочтителен deploy key:

```
ssh-keygen -t ed25519 -C "symbolika-server" -f ~/.ssh/symbolika_deploy
cat ~/.ssh/symbolika_deploy.pub
```

Добавьте публичный ключ в GitHub → репозиторий → `Settings` → `Deploy keys` без права записи. Настройте `~/.ssh/config` и клонируйте по SSH. Не вставляйте GitHub-токен в URL remote.

```
git clone git@github.com:MaximRemizovFEdev/directus_symb_by_vps.git
cd directus_symb_by_vps/symbolika_directus_clean_install
```

Для обновления существующей установки:

```
cd /opt/symbolika/directus_symb_by_vps
git fetch origin
git checkout dev-v1
git pull --ff-only origin dev-v1
```

Перед `pull` серверный рабочий каталог должен быть чистым. `.env`, `database`, `uploads` и дампы не должны отслеживаться Git.

Обновление одной командой

После первоначальной установки систему на VPS `82.146.53.84` можно обновлять с рабочего компьютера одной командой из корня репозитория:

```
.\update-vps.cmd
```

По умолчанию используются:

- SSH-пользователь `root` ;
- сервер `82.146.53.84` ;
- каталог `/opt/symbolika/directus_symb_by_vps` ;
- ветка `dev-v1` .

Если SSH-пользователь или каталог отличаются:

```
.\update-vps.cmd -User deploy -RemoteRepository /opt/symbolika/directus_symb_by_vps
```

Сценарий самостоятельно:

1. останавливается, если на VPS есть незакоммиченные изменения;
2. получает только fast-forward обновление из `origin/dev-v1` ;
3. перед изменением кода создаёт и проверяет PostgreSQL dump в `/opt/symbolika/backups` ;
4. устанавливает зависимости расширений из lock-файлов;
5. применяет `setup/create-work-views.sql` с `ON_ERROR_STOP=1` ;
6. пересоздаёт Directus с серверным `.env` ;
7. ждёт успешного `/server/health` и показывает состояние контейнеров и последние логи.

Команда устанавливает только уже закоммиченный и отправленный в GitHub код. Локальные незакоммиченные изменения на VPS не передаются.

Перед первым запуском один раз проверьте обычное подключение и сохраните SSH host key:

```
ssh root@82.146.53.84
```

Для действительно беспарольного обновления добавьте публичный SSH-ключ рабочего компьютера в `/root/.ssh/authorized_keys` на VPS. Пароли и токены сценарий не копирует: серверный `.env` остаётся на VPS.

4. Передать дамп базы

На рабочем компьютере создайте и проверьте custom dump:

```
docker exec symbolika-db pg_dump -U directus -d directus -Fc -f /tmp/directus.dump
docker cp symbolika-db:/tmp/directus.dump backups/directus-release.dump
pg_restore --list backups/directus-release.dump
```

Передайте его по SCP:

```
scp backups/directus-release.dump user@SERVER_IP:/opt/symbolika/directus-release.dump
```

Дамп не добавлять в Git.

5. Передать локальные uploads при необходимости

```
tar -czf uploads-release.tar.gz -C symbolika_directus_clean_install uploads
scp uploads-release.tar.gz user@SERVER_IP:/opt/symbolika/
```

На сервере распакуйте только в каталог проекта и проверьте путь:

```
cd /opt/symbolika/directus_symb_by_vps/symbolika_directus_clean_install
tar -xzf /opt/symbolika/uploads-release.tar.gz
```

6. После загрузки

1. Создайте `.env` по инструкции [Токены и секреты](#).
2. Выполните [Первоначальную настройку сервера](#).
3. После проверки удалите переданный дамп из общего каталога либо перенесите в защищенное хранилище с ограниченными правами.

Полная инструкция для администратора

1. Назначение

Администратор отвечает за конфигурацию, доступы, справочники, контроль данных, интеграции, резервное копирование и выпуск обновлений. Повседневные операции следует выполнять в рабочих модулях. Стандартные коллекции Directus — технический нижний уровень для диагностики и редких исправлений.

Администратор видит все заказы, позиции, задачи, клиентов, участки, закупки, финансы, события и системные экраны.

2. Архитектура

Система построена на Directus 11 и PostgreSQL/PostGIS.

Ключевые части:

- `setup/create-work-views.sql` — таблицы, представления, роли, права, metadata, функции и триггеры;
- `extensions/symbolika-costing-module/` — основной рабочий интерфейс;
- `extensions/symbolika-calculations/` — расчеты, оплаты, статусы и автоматизации;
- `extensions/symbolika-tasks-module/` — задачи;
- `extensions/symbolika-event-rollback/` — безопасный откат событий;
- `extensions/symbolika-profile/` — профиль и расчетные листы;
- `extensions/symbolika-notifications-module/` и `symbolika-push` — центр и доставка уведомлений;
- `extensions/symbolika-mail/` — IMAP/SMTP почта;
- `extensions/symbolika-yandex-disk/` — загрузка макетов;
- `extensions/symbolika-public-item/` — публичная карточка позиции;
- `extensions/symbolika-support/` — сообщения об ошибках и здоровье автоматизаций.

Рабочий локальный адрес: `http://localhost:8057/admin/`.

3. Карта разделов

Заказы

Сводка, события, проблемы, поиск, все заказы, сроки, собственные заказы, расчеты, клиенты, компании, сверки, клиентские операции, сертификаты и офис. Архивные заказы и позиции выводятся в общем списке переключателем `Активные / Архив / Все` или фильтром по статусу. Заказ содержит позиции; позиция является главным объектом производства, ТЗ, макета, себестоимости и маршрутизации.

В режиме `По заказам` архивность определяется терминальным статусом заказа (`Доставлен/Выдан` или `Отменён`). В режиме `По позициям` используется статус каждой позиции. Выбор конкретного статуса в фильтрах имеет приоритет над переключателем архива.

Задачи

Отдельный таск-трекер с фильтрами, архивом, канбаном, чек-листами, обсуждением, вложениями и статусом ожидания.

Производство

Очереди производства и шелкографии, этикетки, склад и расходка. Готовые и отмененные позиции открываются переключателем **Архив** непосредственно в очереди участка. Пользователь участка видит только свои позиции.

В производстве режим **Активные** скрывает позиции со статусами **Готов** и **Отменён**; **Архив** показывает только их; **Все** объединяет обе группы. Фильтр по конкретному производственному статусу ищет по всей доступной очереди.

Закупки

Единый список исходных заявок и сводных групп. Это каноническое место; историческую витрину закупки не возвращать в меню.

Управление

Себестоимость, уведомления клиентам и контроль автоматизаций.

Клиенты и расчёты внутри заказов

Клиенты и компании, карточки, балансы, контакты, передача менеджера, сверки, клиентские операции и подарочные сертификаты собраны в модуле **Заказы**. Отдельные иконки **Клиенты** и **Финансы** в главном меню не используются.

Админка

Сотрудники, пользователи и роли, должности, контрагенты, категории, подкатегории, виды нанесения, маршрутизация, статусы заказов и производства, финансовый дэшборд, зарплаты, расходы, расчеты с контрагентами и финрезультат.

Отдельными иконками доступны личный кабинет, уведомления и почта. Кабинет контрагента скрыт для обычных внутренних ролей.

4. Пользователь, сотрудник, роль и должность

Это разные сущности:

- пользователь Directus отвечает за вход и роль доступа;
- сотрудник хранит ФИО, контакты, процент, зарплатные и рабочие данные;
- поле `directus_user` связывает сотрудника с учетной записью;
- роль задает права API;
- должность описывает рабочую функцию в справочнике.

Создание сотрудника

1. Создайте пользователя и назначьте роль.

2. Создайте или откройте карточку сотрудника.
3. Свяжите ее с пользователем.
4. Назначьте должность, контакты, активность и процент от заказов.
5. Проверьте вход под этой ролью.

Для дизайнера обязательны роль **Дизайнер** и должность **Дизайнер**. Автозадача назначается первому связанному сотруднику этой роли; при нескольких дизайнерах задачу можно переназначить.

Администратор и управляющий могут быть менеджерами заказа: они должны иметь активную карточку сотрудника, связанную с пользователем.

При отпуске или увольнении сначала передайте клиентов/компании, затем деактивируйте сотрудника и пользователя. Передача меняет текущего ответственного во всех заказах, но не комиссионного менеджера уже созданных продаж.

5. Права ролей

- **Administrator** — полный рабочий и технический доступ.
- **Управляющий** — общая операционная картина, производство, закупки, себестоимость и финансы без технического управления пользователями.
- **Менеджер** — свои клиенты, компании и заказы, расчеты, оплаты и офис.
- **Офис-менеджер** — выдача, офисные статусы и прием оплаты.
- **Производство** — свои производственные позиции, склад и заявки.
- **Шелкография** — свои позиции участка, склад и заявки.
- **Дизайнер** — дизайнерские задачи и отмеченные позиции.
- **Контрагент** — задел отдельного кабинета, не основной внутренний процесс.

После изменения прав проверяйте не только меню, но и прямой API-доступ. Скрытая вкладка сама по себе не является защитой.

6. Справочники и маршрутизация

Администратор поддерживает:

- должности;
- контрагентов и поставщиков;
- категории и подкатегории;
- виды нанесения;
- статусы заказа и производства;
- правила маршрутизации.

Источник истины маршрута:

категория / подкатегория / вид нанесения → подрядчик 1 / подрядчик 2

Исторические поля контрагента с категорией по умолчанию не использовать как основной механизм.

У контрагента настройте сайт, контакты, город, адрес/ПВЗ, способ и срок получения, тип поставщика и признак собственного производства. Сайт открывается из общего списка.

Дополнительное направление на внутренний участок назначается в конкретной позиции. Оно не меняет контрагентов и себестоимость.

7. Заказ и его жизненный цикл

До запуска доступны **Новый** и **Согласование**. Запуск выполняется кнопкой **Отправить в работу**, когда серверная функция готовности подтверждает наличие позиций и обязательных менеджерских полей.

Первый запуск переводит заказ и позиции в **В работе**. После запроса доработки старая ссылка макета запоминается; повторный запуск разрешается после ее изменения и затрагивает только исправленные позиции.

Итоговые связи:

- все позиции готовы ↔ заказ готов;
- все позиции выданы/доставлены ↔ заказ выдан/доставлен;
- отмена распространяется между заказом и всеми позициями;
- производство **Доработка макета** → позиция в доработке;
- производство **Готов** → позиция готова;
- офис **В офисе / Выдан** → производство готово.

Менеджерская готовность к запуску и администраторское **Заполнение** — разные показатели.

Заполнение дополнительно контролирует подрядчиков, себестоимость и условные справочники каждой позиции.

8. Дизайнерская автоматизация

Галочка помощи дизайнера создает одну активную задачу типа **design**. В нее переносятся комментарий, исходная ссылка, срок и связи.

При снятии галочки задача отменяется. При изменении постановки существующая задача обновляется.

Задачу нельзя закрыть без **result_url**. При первом переходе в **Готово** результат записывается в макет позиции, менеджер получает уведомление.

Если нужна повторная доработка после завершения, создайте понятную новую постановку или переоткройте работу; не затирайте историю без причины.

9. Производство и офис

Позиция попадает в очередь участка после запуска и назначения участка основным контрагентом либо внутренним маршрутом.

Офис работает с состояниями **К выдаче**, **В офисе**, **Выданные**. В выдаче показываются позиции и финансы. Оплата из окна выдачи сохраняется без потери контекста, затем выполняется выдача.

Автоматическое клиентское уведомление создается только для полностью готового заказа в офисе со способом `office_pickup`.

10. Склад и закупки

Если остаток активной складской позиции строго меньше минимума, создается одна открытая заявка на разницу. Пока она `Нужно заказать`, изменение остатка пересчитывает количество, а восстановление запаса отменяет автозаявку. Заказанная заявка автоматически не отменяется.

Заявки поступают из склада, заготовок заказов и ручного ввода любого сотрудника. Они объединяются по поставщику и одинаковым условиям доставки. Одинаковые позиции суммируются визуально, но исходные строки сохраняются.

Одна групповая задача управляет пакетом. Ее завершение переводит пакет и заявки в `Заказано`. Без поставщика заказать группу нельзя. После заказа новые заявки образуют новый пакет.

Проверяйте три связи: исходная заявка → пакет → задача.

11. Оплаты и взаиморасчеты

Сумма заказа считается из позиций. Оплачено — из распределений платежей. Частичная оплата распределяется сверху вниз.

При выбранной компании плательщик — компания; иначе клиент.

Начальные остатки импортированных клиентов/компаний создают одну подтвержденную клиентскую операцию типа `opening_balance`. Нулевая сумма убирает финансовый эффект.

Нестандартные просьбы оформляются в `customer_operations`. Только `confirmed` влияет на баланс и не влияет на выручку или процент.

Подарочные сертификаты имеют уникальный код, номинал, остаток, срок, клиента и журнал операций.

12. Себестоимость и финансы

Себестоимость позиции складывается из подрядчиков и применимых заготовок. Для собственного производства подрядная себестоимость всегда нулевая.

Материалы, производственная расходка, обслуживание техники и прочие запасы оформляются операционными расходами и уменьшают чистую прибыль месяца.

Оплата контрагенту закрывает кредиторскую задолженность и не является вторым расходом прибыли.

В расчетах с контрагентами доступны сводка и детализация по заказам. Платеж по заказу автоматически подставляет остаток; возможна частичная сумма.

Зарплата: оклад + процент от оплаченной базы + премии – выплаты/авансы. Процент относится к `commission_manager_employee`. Премия требует сотрудника и основания и уменьшает результат месяца.

13. Задачи и события

Автозадачи создаются для закупок, дизайна и других рабочих условий. У задачи есть связи, чек-лист, комментарии, вложения и архив.

Лента строится на штатном аудите Directus. В карточке заказа видны события заказа, позиций и связанных задач. Откат выполняется endpoint с правами текущего пользователя и создает новое событие.

Перед последовательным откатом оцените все более поздние изменения. Финансовые служебные поля в безопасную дельту не входят.

14. Центр уведомлений

Все внутренние сообщения хранятся в `directus_notifications`; центр не дублирует штатную таблицу. Колокольчик открывает единый центр.

Пользователь выбирает темы: статусы заказов, позиции/макеты, новые и измененные задачи, производство, закупки, почта и финансы. Затем выбирает внешние каналы: push, email, VK, Telegram. Внутренняя история обязательна; критические сообщения не отключаются.

Попытки доставки пишутся отдельно, поэтому сбой провайдера не блокирует бизнес-автоматику.

Полная настройка: [уведомления сотрудников](#). Клиентские каналы: [уведомления клиентов](#).

15. Почта

Модуль поддерживает безопасный тестовый режим и IMAP/SMTP REG.RU. Папки назначаются сотрудникам, псевдонимы — отправителям. Администратор и управляющий видят все папки.

Автообновление выполняется каждые 10 секунд. Письмо автоматически связывается по email и номеру заказа в теме; связь можно исправить вручную. Доступны задачи, счета на оплату, пересылка, теги, рабочие папки и подписи.

Перемещение по рабочим папкам не выполняет IMAP MOVE. Полная настройка: [настройка встроенной почты](#).

16. Яндекс Диск

Токен хранится только в `.env`; приложению нужны `cloud_api:disk.app_folder` и `cloud_api:disk.info`.

Endpoint принимает загрузку до 2 ГБ. Имя создается как `ДДММГГ_Заказчик_Позиция - Количествошт.ext`. При успешной замене старый управляемый файл удаляется, внешние ссылки не затрагиваются.

Проверка: `GET /symbolika-yandex-disk/status` должна вернуть `configured: true`, `connected: true`.

Подробности: [настройка Яндекс Диска](#). Если описание структуры имен в старом документе расходится с кодом, источником истины является текущий endpoint.

17. Публичные ссылки и клиентские уведомления

У каждой позиции постоянный публичный токен. Авторизованный пользователь открывает рабочую карточку, внешний — безопасную страницу без ТЗ, цены и внутренних данных.

В Excel этикеток выводится ссылка; QR формируется принтером или шаблоном.

Для клиента выбирается один основной канал. Сообщение готовности содержит позиции, количество, цены, итог, оплачено, остаток, адрес и время офиса. Отправка защищена от дублей.

18. Темы, мобильная версия и UI

Доступны темы `Графит`, `Эспрессо`, `Жемчуг`, `Иней`. Пользователь выбирает тему в профиле.

На мобильных экранах показывается минимальный набор заказа и позиции, сохраняется быстрое создание заказа. Проверяйте длинные названия, dropdown, календарь, карточки, таблицы и модальные окна во всех темах.

Новое модальное окно должно быть выше предыдущего. Закрытие вложенной позиции возвращает на исходный экран, а не всегда в заказ.

19. Контроль автоматизаций и обратная связь

Экран контролирует несогласованные статусы, закупки без задач и выполненные задачи с незакрытыми закупками. Дополнительно показывает последнее выполнение обработчиков и ошибки уведомлений.

`Сообщить об ошибке` сохраняет пользователя, URL, раздел, сущность и браузер. Администратор меняет состояние обработки обращения.

Безопасный повтор запускает штатную проверку/доставку с защитой от дублей. Не используйте ручные SQL-правки до анализа причины.

20. Резервное копирование

Живую папку `database/` не коммитить. Для переноса использовать только custom dump:

```
docker exec symbolika-db pg_dump -U directus -d directus -Fc -f /tmp/directus.dump
docker cp symbolika-db:/tmp/directus.dump backups/directus.dump
pg_restore --list backups/directus.dump
```

Храните минимум одну проверенную копию вне сервера. Перед миграцией, SQL-обновлением и релизом делайте новый дамп.

21. Выпуск обновления

После коммита, push и релизной проверки обновление VPS `82.146.53.84` запускается из корня рабочего репозитория одной командой:

```
.\update-vps.cmd
```

Команда проверяет чистоту серверного репозитория, создаёт проверенный dump базы, получает только fast-forward ветки `dev-v1`, обновляет зависимости, применяет SQL, пересоздаёт Directus и ждёт успешного health-check. Серверный `.env` она не копирует и не изменяет. Подробности: [Загрузка релиза на сервер](#).

Перед отправкой релиза в Git выполните проверки ниже.

1. Проверить `git status --short` ; не включать секреты, базу, дампы и временные файлы.
2. Проверить JS:

```
node --check symbolika_directus_clean_install\extensions\symbolika-costing-module\index.js
node --check symbolika_directus_clean_install\extensions\symbolika-calculations\index.js
node --check symbolika_directus_clean_install\setup\symbolika-admin-ui.js
```

1. Сделать custom dump и проверить `pg_restore --list` .
2. Применить SQL:

```
cmd /c "docker exec -i symbolika-db psql -v ON_ERROR_STOP=1 -U directus -d directus <
symbolika_directus_clean_install\setup\create-work-views.sql"
```

1. Перезапустить и проверить логи:

```
docker restart symbolika-directus
docker logs --tail 80 symbolika-directus
```

1. Прогнать Playwright под админом, управляющим, менеджером, производством, шелкографией, офисом и дизайнером.
2. Проверить создание заказа, запуск, доработку, готовность, офис, оплату, выдачу, закупку, дизайнерский результат и уведомление.
3. На Linux проверить LF у `*.sh` .

22. Регулярный чек-лист администратора

Ежедневно:

- здоровье автоматизаций и ошибки доставки;
- сообщения сотрудников;
- критические рассинхронизации;
- логи после обновлений.

Еженедельно:

- сотрудники без пользователя и пользователи без сотрудника;
- просроченные задачи и зависшие закупки;
- пустые себестоимости и маршруты;
- складские минимумы;
- долги клиентов и контрагентов;
- свободное место базы и файлового хранилища.

Ежемесячно:

- зарплаты, премии и расчетные листы;

- расходы и финансовый результат;
- проверенный резервный дамп;
- тест восстановления;
- аудит активных пользователей, ролей и внешних токенов.

23. Диагностика типовых проблем

Пользователь не видит раздел

Проверить роль пользователя, связь с сотрудником, активность сотрудника, политику роли и список вкладок модуля.

Заказ не запускается

Открыть детали готовности и проверить наличие позиции, наименование, количество, ТЗ и макет. Проверить серверную функцию готовности.

Позиция не видна участку

Проверить рабочий статус, контрагентов, правило маршрутизации и дополнительный внутренний маршрут.

Сумма или оплачено неверны

Проверить позиции, платежи и распределения. Не редактировать итог вручную.

Закупочная задача закрыта, статус не изменился

Проверить тип задачи, связь с пакетом, состав пакета и `Контроль автоматизаций`; затем выполнить безопасный повтор.

Макет дизайнера не перенесся

Проверить тип задачи, связанную позицию, статус `Готово` и заполненный `result_url`.

Не отправилось уведомление

Проверить пользовательскую тему события, канал, идентификатор, переменные окружения, журнал доставок и здоровье обработчика.

После переноса на Linux падает shell-скрипт

Исправить CRLF:

```
find setup -name "*.sh" -exec sed -i 's/\r$//' {} \;
```

24. Главный принцип исправлений

Расчетные суммы, итоговые статусы и системные связи восстанавливаются через исходные данные, штатные функции, безопасный повтор или откат события. Прямую правку базы применять только

после резервной копии, установления причины и проверки затрагиваемых объектов.

Инструкция для дизайнера

1. Назначение роли

Дизайнер работает с задачами типа **Макет / дизайн**, которые создаются для позиций с галочкой **Нужна помощь дизайнера**.

Для доступа сотруднику необходимы одновременно:

- пользователь Directus с ролью **Дизайнер**;
- карточка сотрудника с должностью **Дизайнер**, связанная с этим пользователем.

Дизайнер видит дизайнерские задачи, связанные позиции, доступную историю событий и собственные заказы. Финансовые и административные разделы ему не показываются.

2. Как создается задача

Менеджер или производство в позиции указывает:

- **Нужна помощь дизайнера**;
- комментарий для дизайнера;
- исходный файл или ссылку;
- срок позиции.

Система создает одну активную дизайнерскую задачу по позиции и связывает с ней заказ, клиента, компанию и срок. По умолчанию она назначается первому активному сотруднику с ролью дизайнера; администратор или управляющий может изменить исполнителя.

Если галочку снять, активная задача отменяется. Изменение комментария, исходной ссылки или срока обновляет существующую задачу, а не создает дубль.

3. Рабочий процесс

1. Откройте **Задачи** и выберите новую дизайнерскую задачу.
2. Проверьте заказ, позицию, количество, ТЗ, срок и постановочный комментарий.
3. Откройте **Исходный файл / ссылка**.
4. Если информации недостаточно, напишите вопрос в обсуждении и поставьте **Жду ответа**.
5. Переведите задачу в работу.
6. Используйте чек-лист для нескольких этапов.
7. Приложите промежуточные материалы во вложения, если это удобно.
8. Итоговую ссылку обязательно внесите в **Ссылка на готовый макет**.
9. После проверки поставьте статус **Готово**.

Исходный файл не является готовым результатом. Вложения также не заменяют поле результата.

4. Статусы

- **Новая** — задача еще не принята.
- **В работе** — дизайнер выполняет макет.
- **Жду ответа** — нужен ответ постановщика или участника.
- **Нужны правки** — результат возвращен на корректировку.
- **Готово** — итоговый макет передан.
- **Отменена** — помощь больше не нужна.

Без ссылки на готовый макет завершение дизайнерской задачи блокируется.

5. Что происходит после завершения

Когда задача впервые переходит в **Готово** :

- ссылка из результата автоматически записывается в поле макета позиции;
- менеджер заказа получает уведомление **Макет готов** ;
- позицию можно проверить и отправить в работу.

Дизайнеру не нужно вручную менять статус заказа или производства.

Если после запуска производство запросило новую доработку, работа должна быть оформлена повторным открытием/новой постановкой дизайнерской задачи с актуальным комментарием и исходником. Не заменяйте завершённый результат без согласованной постановки: история должна объяснять причину новой версии.

6. Обсуждения и файлы

Обсуждение хранит автора и время сообщения. Используйте его для уточнений и согласования, а не личные сообщения вне системы — так история останется в задаче.

Вложения:

- до 25 МБ на файл;
- до 10 файлов за один выбор;
- подходят для референсов и промежуточных материалов;
- итоговая ссылка все равно указывается отдельно.

7. Лента и уведомления

Дизайнер видит события дизайнерских позиций, своих задач и задач, которые он поставил. В **Центре уведомлений** можно выбрать категории событий и внешние каналы.

Внутрисистемная история остается включенной всегда. Для оперативной работы рекомендуется оставить включенными **Новые задачи** , **Изменения задач** и **Позиции и макеты** .

8. Собственные заказы

Дизайнер, как и любой активный сотрудник, может создать собственный заказ в **Мои заказы** и получать с него процент по ставке своей карточки сотрудника. Это отдельная роль в конкретной продаже и не дает доступа к чужим менеджерским заказам.

9. Короткая памятка

1. Начинайте с задач и уведомлений.
2. Всегда читайте ТЗ и комментарий постановщика.
3. При нехватке данных используйте обсуждение и **Жду ответа**.
4. Исходник и результат храните в разных полях.
5. Перед **Готово** проверьте итоговую ссылку.
6. Не меняйте статусы заказа и производства.

Инструкция для менеджера

1. Зона ответственности

Менеджер ведет клиента от обращения до выдачи заказа:

- создает клиентов, компании, расчеты, заказы и позиции;
- собирает исходные данные, ТЗ и макеты;
- контролирует согласование, сроки, производство и доработки;
- принимает и проверяет оплаты;
- передает готовый заказ клиенту;
- ведет задачи, переписку и уведомления по своим заказам.

Менеджер видит свои заказы, клиентов и компании. В офисном разделе доступ может быть шире, чтобы сотрудник мог выдать заказ, который физически находится в офисе.

Любой активный сотрудник может создать собственный заказ в [Заказы → Мои заказы](#). Автор становится ответственным и комиссионным менеджером этого заказа. Процент начисляется по ставке в карточке сотрудника.

2. Основные разделы

Заказы

- [Сводка](#) — показатели и то, что требует внимания.
- [Лента событий](#) — история доступных заказов, позиций и задач.
- [Требуется внимания](#) — просрочки и проблемные рабочие состояния.
- [Поиск](#) — поиск связанных объектов.
- [Сроки](#) — заказы по временным периодам.
- [Мои заказы](#) — основная рабочая страница.
- [Расчеты](#) — предварительные предложения клиентам.
- Переключатель [Активные](#) / [Архив](#) / [Все](#) в списке заказов — текущие, завершённые и отменённые заказы без перехода на отдельную страницу. Те же записи можно выбрать фильтром по статусу.
- В режиме [По заказам](#) архив определяется статусом заказа. В режиме [По позициям](#) — статусом конкретной позиции, поэтому завершённую позицию можно найти, даже если остальные позиции заказа ещё выполняются.
- [Офис](#) — готовые к выдаче, находящиеся в офисе и выданные заказы.

Другие разделы

- [Задачи](#) — все личные и поставленные задачи, обсуждения и чек-листы.
- Внутри [Заказов](#) находятся клиенты, компании, сверки, клиентские операции и подарочные сертификаты.
- [Закупки](#) — создание и просмотр доступных закупочных заявок.
- [Почта](#) — рабочая почта, если сотруднику назначена папка.

- **Центр уведомлений** — персональная лента и настройки доставки.
- **Личный кабинет** — контакты, аватар, тема, уведомления и собственный расчетный лист.

3. Клиенты и компании

Клиент — конкретное физическое лицо. Компания — организация, от которой он может размещать заказ.

При личном заказе выбирается только клиент. Его долг и оплаты учитываются в личной сверке.

При заказе от организации выбираются клиент и компания. Плательщиком становится компания, а клиент остается контактным лицом. Связь клиента с компанией создается автоматически.

При создании проверьте:

- имя или название;
- телефон и email;
- основной канал клиентского уведомления;
- идентификатор канала, если выбран VK, Telegram или SMS;
- ответственного менеджера;
- комментарий и реквизиты, если они нужны.

Начальный баланс при переносе из старой системы вводится специальными полями начального остатка. Сумма указывается положительной, направление выбирается отдельно: клиент должен нам либо мы должны клиенту. Начальный остаток участвует в сверке, но не считается заказом, оплатой, выручкой или базой процента.

Смену ответственного менеджера выполняют администратор или управляющий. Текущие и архивные заказы перейдут новому ответственному, но комиссия по уже созданным заказам останется у их автора.

4. Расчет до заказа

Расчет используется, когда клиенту нужно предварительное предложение без полноценной производственной карточки.

1. Откройте **Заказы** → **Расчеты**.
2. Выберите клиента и при необходимости компанию.
3. Добавьте позиции, количество и цену.
4. Сохраните расчет или выгрузите его.
5. После согласования сформируйте заказ из расчета.

Категории, производственная маршрутизация и детальное ТЗ заполняются уже в заказе.

5. Создание заказа

1. Откройте **Мои заказы** и нажмите **Новый заказ**.
2. Выберите существующего клиента или создайте нового.
3. Если заказ от компании, выберите компанию.

4. Укажите дату, срок, способ получения и тип оплаты.
5. Добавьте одну или несколько позиций.
6. Сохраните заказ.

Номер заказа создается автоматически. Дата по умолчанию — текущая. Срок позиции наследуется из срока заказа, но его можно изменить.

Незавершенная форма нового заказа сохраняется в браузере на 30 дней отдельно для пользователя. После обновления страницы система предложит восстановить черновик. После успешного создания или явного отказа черновик удаляется.

Клик вне карточки создания не должен закрывать форму и уничтожать введенные данные. Закрывайте форму крестиком или явной кнопкой отмены.

6. Позиция заказа

Позиция — главный рабочий объект. Для запуска в работу менеджеру необходимо заполнить:

- наименование;
- количество;
- ТЗ;
- макет.

Цена может быть указана позднее. Категория, подкатегория, вид нанесения, подрядчики и себестоимость нужны для полноценного учета и администраторского показателя заполнения, но не блокируют первичный запуск по менеджерскому правилу, если серверная проверка готовности не требует их для конкретной конфигурации.

Дополнительно укажите:

- срок позиции;
- цену за единицу;
- категорию, подкатегорию и вид нанесения;
- способ получения;
- источник заготовки;
- комментарии производству и дизайнеру;
- внутренний маршрут, если его назначает управляющий или администратор.

Сумма позиции и сумма заказа считаются автоматически.

Источник заготовки

- **У поставщика** — укажите поставщика; система создает закупочную заявку.
- **Заготовка заказчика** — закупка не создается.
- **Со склада** — поставщик не нужен, заявка на закупку не создается.

7. Макет и Яндекс Диск

Макет загружается кнопкой или перетаскиванием файла в окно загрузки. В форме нового заказа файл сначала прикрепляется к черновику. После сохранения позиции он отправляется на Яндекс Диск, а в позиции сохраняется ссылка.

Пользователь не редактирует ссылку вручную в форме. В сохраненной карточке ее можно открыть, а заменить файл — повторной загрузкой.

Имя файла формируется по шаблону:

```
ДДММГГ, Заказчик, Позиция - Количествошт.ext
```

При замене система сначала успешно загружает новый файл и обновляет позицию, затем удаляет прежний управляемый файл с Диска. Внешние ссылки, не созданные этой интеграцией, система не удаляет.

Не закрывайте карточку, пока индикатор загрузки не завершился и сохранение не подтвердилось.

8. Запуск в работу и статусы

До запуска менеджер работает со статусами **Новый** и **Согласование**. Рабочий статус не выбирается вручную.

Кнопка **Отправить в работу** появляется, когда:

- в заказе есть хотя бы одна позиция;
- все позиции прошли серверную проверку менеджерских обязательных полей.

При нажатии заказ и готовые к запуску позиции переходят в **В работе**. После запуска итоговыми статусами менеджера являются **Готов** и **Доставлен**.

Связь статусов:

- все позиции готовы → заказ становится **Готов**;
- заказ поставлен **Готов** → все позиции становятся готовыми;
- все позиции выданы/доставлены → заказ становится **Доставлен / Выдан**;
- заказ выдан/доставлен → все позиции получают тот же итог;
- отмена заказа распространяется на позиции;
- статус производства **Готов** делает позицию готовой;
- офисный статус **В офисе** или **Выдан** принудительно делает производственный статус готовым.

Доработка макета

Если производство ставит **Доработка макета**, позиция получает тот же статус. Система запоминает старую ссылку. После загрузки нового макета снова появляется **Отправить в работу**. Повторно запускаются только позиции с правками; уже готовые позиции не откатываются.

9. Помощь дизайнера

В позиции включите **Нужна помощь дизайнера** и заполните:

- комментарий для дизайнера;
- исходный файл или ссылку.

Система создаст задачу типа **Макет / дизайн**, привяжет заказ, позицию, клиента, компанию и срок. Исходник и готовый результат — разные поля.

Дизайнер ведет обсуждение, может прикладывать файлы и указывает **Ссылку на готовый макет**. Без результата задача не должна завершаться. При статусе **Готово** ссылка автоматически переносится в макет позиции, а менеджеру приходит уведомление.

После получения результата проверьте ТЗ и макет, затем запустите позицию в работу.

10. Задачи

Работайте с задачами только в отдельном разделе **Задачи**.

В задаче доступны:

- исполнитель, постановщик, срок и приоритет;
- связь с заказом и позицией;
- чек-лист;
- обсуждение;
- вложения до 25 МБ каждое, не более 10 за один выбор;
- статус **Жду ответа**.

Повторное нажатие **Жду ответа** возвращает задачу в работу. Архив содержит закрытые задачи и поддерживает фильтры и сортировку.

11. Закупочные заявки

Менеджер может создать заявку вручную, например для покупки на маркетплейсе. Укажите:

- что купить;
- количество и единицу;
- где купить или поставщика;
- ссылку на товар, если есть;
- участок, условия получения и комментарий.

Заготовка **У поставщика** создает заявку автоматически. Заявки объединяются по одному поставщику и одинаковым условиям получения. Одинаковые позиции суммируются визуально, но связь с каждым исходным заказом сохраняется.

Статус группы меняют администратор или управляющий. Завершение общей задачи закупки переводит пакет и входящие заявки в **Заказано**.

12. Оплаты, сверка и сертификаты

Система считает:

- сумму заказа — по позициям;

- оплачено — по распределенным платежам;
- остаток — сумма минус оплачено.

Частичная оплата распределяется по позициям сверху вниз. Если платеж создан, но не распределен, долг заказа не уменьшится.

В **Заказы → Клиенты и расчёты → Сверки** проверяйте взаиморасчеты. Личные заказы относятся к клиенту, заказы от компании — к компании.

Нестандартные просьбы клиента оформляются как **Клиентская операция**, а не фиктивный заказ. Только подтвержденная операция влияет на баланс; она не является выручкой и не дает менеджерский процент.

Подарочный сертификат можно привязать к клиенту и списать по заказу. Система хранит номинал, остаток, срок действия и журнал списаний/возвратов.

13. Выдача заказа

Когда заказ готов, появляется кнопка **Выдать клиенту**.

В окне выдачи проверьте:

- получателя и контакты;
- менеджера;
- способ получения;
- все готовые позиции;
- сумму, оплачено и остаток;
- оплату при получении.

Если нужна оплата, нажмите **Добавить оплату**. После сохранения оплаты система возвращает в окно выдачи. Затем нажмите **Выдать**. Заказ и позиции получают итоговый статус **Доставлен / Выдан** и уйдут в архив активной выдачи.

Автоматическое уведомление клиенту отправляется только когда заказ полностью готов, находится в офисе и выбран способ **Выдача в офисе**. Для доставки автоматическое клиентское сообщение пока не отправляется.

14. Почта и уведомления

Почта связывает письмо с клиентом и компанией по email, а с заказом — по номеру вида **S0-00012** в теме. Связь можно исправить вручную кнопкой **Привязать**.

Из письма можно:

- создать задачу;
- создать задачи администратору и управляющему по счету на оплату;
- переслать письмо;
- назначить теги;
- переложить письмо в рабочую папку системы.

В Центре уведомлений выберите события и внешние каналы. Внутрисистемная история обязательна. Критические сообщения отключить нельзя.

15. Ежедневная памятка

В начале дня:

1. Откройте сводку, задачи и уведомления.
2. Проверьте горящие сроки и доработки макетов.
3. Проверьте неоплаченные заказы и заказы в офисе.

По каждому заказу:

1. Заполните клиента и позиции.
2. Проверьте ТЗ и загруженный макет.
3. При необходимости создайте дизайнерскую или закупочную задачу.
4. Нажмите **Отправить в работу**.
5. Следите за сроком и комментариями участка.
6. Примите оплату и выдайте заказ.

Если расчет, статус или связь выглядят неверно, не редактируйте служебное поле вручную. Нажмите **Сообщить об ошибке** и опишите ожидаемый результат.

Инструкция для производства и шелкографии

1. Зона ответственности

Производство и шелкография выполняют только позиции, направленные на соответствующий участок. Главный рабочий объект — позиция заказа, а не заказ целиком.

Сотрудник участка:

- видит свою очередь и архив;
- проверяет срок, количество, ТЗ и макет;
- меняет производственный статус;
- пишет производственный комментарий;
- создает задачи и закупочные заявки;
- работает со складом и этикетками в пределах прав.

Цена, прибыль, комиссия менеджера и административная себестоимость не относятся к производственной работе.

2. Основные разделы

- **Производство** или **Шелкография** — активная очередь участка.
- Переключатель **Активные / Архив / Все** в очереди участка — текущие, готовые и отмененные позиции без отдельной страницы архива.
- **Этикетки** — массовая выгрузка этикеток по отфильтрованным позициям.
- **Склад и расходка** — материалы и текущие остатки.
- **Закупки** — ручные заявки и доступные закупочные состояния.
- **Задачи** — личные и связанные задачи.
- **Мои заказы** — собственные заказы сотрудника, если он выступает менеджером своей продажи.
- **Лента событий**, **Центр уведомлений**, **Личный кабинет**.

Производство и шелкография видят только свои рабочие позиции. Администратор и управляющий видят обе очереди.

3. Как позиция попадает на участок

Позиция появляется в очереди только после запуска в работу и при выполнении одного из условий:

- участок назначен основным или вторым контрагентом по правилу маршрутизации;
- администратор или управляющий включил дополнительный внутренний маршрут именно для этой позиции.

Дополнительный маршрут не меняет контрагентов и себестоимость. Одна позиция может одновременно быть видна производству и шелкографии, если этапы выполняют оба участка.

До статуса **В работе** назначенный маршрут сохраняется, но позиция не должна появляться как активная производственная работа.

4. Что проверять перед началом

В строке или карточке позиции проверьте:

- номер и дату заказа;
- заказчика и менеджера;
- наименование и количество;
- срок;
- ТЗ;
- макет;
- комментарий производства;
- текущий статус;
- источник заготовки и связанные закупки, если они важны для выполнения.

Если данных недостаточно, не начинайте производство «по памяти». Создайте задачу менеджеру, напишите комментарий или поставьте статус **Доработка макета**.

5. Производственные статусы

Менять нужно производственный статус позиции. Основная логика:

- рабочий статус показывает, что позиция принята и выполняется;
- **Доработка макета** переводит и статус позиции в доработку;
- **Готов** переводит позицию в **Готов** ;
- когда все позиции готовы, заказ автоматически становится готовым;
- статус офиса **В офисе** или **Выдан** автоматически исправляет забытый производственный статус на **Готов**.

Если позиция уже готова, отдельный производственный статус может скрываться в заказе как лишняя информация.

Не переводите позицию в готовую, если фактически завершен только промежуточный этап другого участка. Используйте комментарий и задачу для передачи следующему участку.

6. Доработка макета и дизайнер

Если макет нельзя использовать:

1. Поставьте **Доработка макета**.
2. Подробно опишите проблему в производственном комментарии.
3. При необходимости включите помощь дизайнера или создайте связанную задачу через менеджера/управляющего.
4. Приложите исходный файл или ссылку.

Менеджер загрузит новую версию. Система сравнивает ее с сохраненной ссылкой и разрешает повторный запуск только после замены макета. Уже готовые позиции заказа не возвращаются в работу.

Для дизайнерской постановки комментарий должен отвечать на три вопроса: что сейчас неверно, что нужно получить и к какому сроку.

7. Комментарии, задачи и обсуждения

Производственный комментарий нужен для краткого состояния самой позиции. Для отдельной работы используйте задачу.

В задаче можно:

- назначить исполнителя и срок;
- связать заказ и позицию;
- составить чек-лист;
- вести обсуждение;
- прикрепить файлы;
- поставить `Жду ответа`.

Закрывайте задачу только после фактического результата. Закрытие закупочной задачи влияет на закупочную автоматику, поэтому ее нельзя использовать как обычную отметку «прочитал».

8. Склад

Для каждой складской позиции хранятся:

- название и тип;
- участок;
- единица измерения;
- остаток;
- минимальный остаток;
- поставщик и место хранения;
- активность.

Если остаток активной позиции становится строго меньше минимума, система создает одну открытую заявку на разницу до минимума.

Пока заявка имеет статус `Нужно заказать`, изменение остатка или минимума пересчитывает количество. Если запас восстановился, такая автозаявка отменяется. Уже заказанную закупку система автоматически не отменяет.

Остаток должен отражать фактическое наличие. Не изменяйте минимум вместо корректировки остатка, чтобы скрыть дефицит.

9. Закупочные заявки

Любой сотрудник участка может создать заявку вручную. Это используется для бумаги, тонера, красок, пленки, химии, инструмента или покупки на маркетплейсе.

Заполните:

- что купить;
- количество и единицу;
- участок;
- поставщика или место покупки;
- ссылку на товар;
- способ получения, город, адрес или ПВЗ;
- комментарий.

Открытые заявки объединяются по поставщику и одинаковым условиям получения. В одной группе могут быть расходка, заготовки разных заказов и ручные просьбы. Одинаковые позиции суммируются в интерфейсе, но исходные связи сохраняются.

Администратор и управляющий оформляют группу и меняют ее статус. После заказа новые заявки того же поставщика попадут уже в новую открытую группу.

10. Этикетки

Этикетки формируются массово для выбранных или всех отфильтрованных позиций. В выгрузке есть заказчик, позиция и ссылка. Из ссылки принтер или внешний шаблон может сформировать QR-код.

Внутренний сотрудник по ссылке после авторизации открывает рабочую карточку позиции. Внешний посетитель видит только безопасную общую информацию, публичный статус, контакты менеджера и ссылки компании — без ТЗ, цены и внутренних данных.

Перед выгрузкой проверьте фильтр: действие **все отфильтрованные** относится ко всему результату фильтра, а не только к загруженной на экране порции.

11. Собственный заказ сотрудника

Если сотрудник участка самостоятельно привел клиента, он может создать заказ в **Мои заказы**. Для такого заказа сотрудник становится ответственным и комиссионным менеджером и получает процент по своей ставке.

Производственный заказ и собственная продажа — разные контексты. В очереди участка меняйте только производственные поля. Данные собственного заказа редактируйте в **Мои заказы**.

12. Ежедневная памятка

1. Проверьте уведомления, задачи и новые позиции.
2. Отсортируйте очередь по сроку.
3. Откройте ТЗ и макет до начала работы.
4. Отметьте рабочий статус и важный комментарий.
5. При нехватке материалов создайте закупочную заявку.

6. При проблеме макета поставьте доработку и опишите причину.
7. По завершении поставьте **Готов**.
8. Проверьте, что позиция исчезла из режима **Активные** и доступна по переключателю **Архив**.

Если позиция не появилась, проверьте через управляющего: запущена ли она в работу, назначен ли ваш участок и не завершена ли она уже.

Инструкция для управляющего

1. Зона ответственности

Управляющий контролирует сквозной процесс: продажи, сроки, производство, закупки, себестоимость, взаиморасчеты и работу автоматизаций. Он видит общую картину и может выполнять большинство операционных действий администратора, но технические пользователи, роли и системный деплой остаются обязанностью администратора.

2. Ежедневные экраны

- **Сводка** и **Требуется внимания** — оперативные отклонения.
- **Все заказы**, **Сроки** — общая картина продаж. Завершённые и отменённые записи открываются в общем списке переключателем **Активные / Архив / Все**.
- **Задачи** — задачи команды и автозадачи.
- **Производство**, **Шелкография** — очереди участков с переключателем **Активные / Архив / Все**.
- **Офис** — готовность к выдаче и остаток оплаты.
- **Управление → Себестоимость** — подрядчики, затраты, прибыль и маржа.
- **Закупки** — единый канонический список закупок.
- **Управление → Контроль автоматизаций** — рассинхронизации, здоровье обработчиков и сообщения об ошибках.
- **Заказы → Клиенты и расчёты** — клиенты, компании, передача ответственного, сверки, клиентские операции и подарочные сертификаты.

3. Контроль заказа

Управляющий видит менеджерский показатель готовности к запуску и расширенный показатель **Заполнение**.

Для запуска менеджеру достаточно обязательных рабочих данных позиции. Администраторское заполнение дополнительно проверяет:

- поля заказа;
- категорию, подкатеорию и нанесение, когда для категории они применимы;
- подрядчиков;
- себестоимость каждого подрядчика;
- готовый макет для дизайнерской позиции;
- прочие обязательные поля всех позиций.

Полоса заполнения показывает процент, а детали незаполненных полей открываются при наведении или взаимодействии.

Управляющий не должен вручную подменять автоматические суммы и итоговые статусы. Нужно исправить исходные позиции или использовать контроль автоматизаций.

4. Статусы и выдача

Проверяйте сквозную связь:

- позиции → итог заказа;
- производство → статус позиции;
- офис → производство и выдача;
- выдача заказа → все его позиции.

Если позиция физически в офисе, ее производство автоматически считается готовым. Если все позиции готовы, заказ готов. После выдачи все позиции и заказ получают итоговый статус.

В окне **Выдать клиенту** отображаются получатель, менеджер, позиции, сумма, оплачено и остаток. При оплате сначала сохраните платеж, затем завершите выдачу.

5. Внутренняя маршрутизация

Основной маршрут определяется правилами:

категория / подкатегория / вид нанесения → подрядчик 1 / подрядчик 2

Если в выполнении дополнительно участвует внутренний участок, откройте конкретную позицию и включите:

- **Передать в производство** ;
- **Передать в шелкографию** .

Маршрут назначается по позиции, а не по заказу. Он не меняет основного контрагента и себестоимость. После запуска позиция появится в очередях выбранных участков.

6. Передача клиента или компании

Управляющий может сменить ответственного менеджера в карточке клиента или компании.

Передача клиента меняет ответственного у его текущих и архивных заказов и позиций. Передача компании также меняет ответственного у связанных клиентов и всех заказов компании.

Комиссионный менеджер уже созданных заказов не меняется. Новый ответственный получает процент только с заказов, которые сам создаст как комиссионный менеджер.

Перед передачей проверьте выбранного сотрудника, потому что действие затрагивает историю и текущую работу.

7. Себестоимость

Для каждой позиции указываются:

- подрядчик 1 и себестоимость 1;
- подрядчик 2 и себестоимость 2;
- заготовка и ее стоимость, если применимо.

Система рассчитывает итоговую себестоимость, прибыль и маржу.

Если контрагент отмечен как **Собственное производство**, его подрядная себестоимость принудительно равна нулю. Материалы, расходка, обслуживание техники и прочие запасы учитываются отдельными операционными расходами месяца.

Оплата контрагенту закрывает долг, но повторно не уменьшает прибыль: себестоимость уже вычтена из заказа.

8. Расчеты с контрагентами

Доступны два режима:

- **По контрагентам** — начислено, оплачено и общий долг;
- **По заказам** — заказ, позиции, контрагент, начислено, оплачено и остаток.

Кнопка оплаты в строке заказа подставляет контрагента, заказ и остаток. Сумму можно уменьшить для частичной оплаты.

Платеж без привязки к заказу влияет на общий баланс контрагента, но не распределяется по заказам автоматически.

9. Закупки

Закупки — единый список. Не нужно искать дубли в заказах, производстве или админке.

Источники заявок:

- закупаемая заготовка позиции;
- ручная заявка менеджера или любого сотрудника;
- заявка производства/шелкографии;
- автоматический дефицит складского остатка.

Открытые заявки объединяются по поставщику и одинаковым условиям получения: способ, транспортная компания, город и адрес. Без поставщика группу заказать нельзя.

В группе сохраняются исходные заявки и их связи. Одинаковые позиции суммируются только в представлении.

Рабочий порядок:

1. Проверить название, количество, ссылку и комментарий.
2. Назначить поставщика и условия получения.
3. Проверить состав сводной группы.
4. Выполнить общую задачу закупки.
5. Убедиться, что группа и заявки стали **Заказано**.
6. После получения поставить следующий фактический статус.

Новая заявка после заказа текущей группы создаст новую открытую группу, даже при том же поставщике.

10. Контроль автоматизаций

Экран показывает:

- заказ с итоговым статусом, не совпадающим с позициями;
- закупку без активной задачи;
- выполненную закупочную задачу с незакрытой заявкой или группой;
- время последней попытки и последнего успешного запуска обработчиков;
- последние ошибки уведомлений;
- сообщения сотрудников через **Сообщить об ошибке**.

Используйте безопасный повтор только после проверки причины. Он запускает штатную проверку или доставку с защитой от дублей, а не произвольно переписывает итоговые данные.

Если ошибка повторяется, зафиксируйте заказ/позицию, время, действие пользователя и текст ошибки и передайте администратору.

11. Лента событий и откат

Управляющий видит всю ленту событий заказов, позиций и задач.

В карточке события доступны варианты:

- откатить только указанное изменение;
- откатить все изменения объекта, включая выбранное событие.

Откат выполняется с текущими правами и сам создает новое событие. Перед подтверждением проверьте последующие действия других сотрудников: последовательный откат может затронуть несколько осмысленных исправлений.

Финансовые служебные поля не хранятся в безопасной дельте ленты и не должны восстанавливаться этим механизмом.

12. Финансы и сотрудники

Управляющий контролирует:

- сверки клиентов и компаний;
- нестандартные клиентские операции;
- подарочные сертификаты;
- результаты менеджеров;
- расчетные листы;
- премии;
- расходы;
- расчеты с контрагентами;
- финансовый результат по месяцам.

Премия оформляется расходом типа **Премия сотруднику** с обязательным сотрудником и основанием. Она входит в начисление и выплату и уменьшает финансовый результат месяца.

Процент менеджера считается по оплаченным заказам комиссионного менеджера, а не текущего ответственного.

13. Почта, счета и задачи

Управляющий видит все почтовые папки.

Из письма можно привязать заказ, клиента и компанию, создать задачу, назначить теги, переслать и обработать счет на оплату. Кнопка **Счет на оплату** создает важные задачи активным администраторам и управляющим и защищена от повторного создания по одному письму.

Рабочее перемещение письма меняет папку в учетной системе, но не выполняет физический IMAP MOVE на сервере.

14. Ежедневный контроль

Утром:

1. Проверить уведомления, задачи и **Требуется внимания**.
2. Проверить горящие сроки и доработки.
3. Проверить очереди участков и офис.
4. Проверить новые закупки и счета.
5. Проверить здоровье автоматизаций.

В конце дня:

1. Проверить просроченные задачи.
2. Сверить готовые позиции с заказами и офисом.
3. Проверить незакрытые закупочные группы.
4. Просмотреть новые сообщения об ошибках.

Еженедельно проверяйте себестоимость незаполненных позиций, долги клиентов и контрагентов, складские минимумы и результаты менеджеров.

Встроенная почта «Символики»

Тестовый режим

По умолчанию модуль запускается с настройкой:

```
SYMBOLIKA_MAIL_MODE=mock
```

В этом режиме тестовые письма хранятся в локальной базе. Кнопки ответа и создания письма работают, но письма не покидают систему. Это позволяет безопасно проверить интерфейс, папки и права доступа.

Подключение REG.RU

После первого переноса новых файлов на сервер установите зависимости endpoint:

```
cd extensions/symbolika-mail  
npm install --omit=dev
```

Пароль почтового ящика не нужно записывать в `docker-compose.yml` и нельзя передавать в чат. Создайте рядом с `docker-compose.yml` файл `.env` и задайте:

```
SYMBOLIKA_MAIL_MODE=imap  
SYMBOLIKA_IMAP_HOST=mail.hosting.reg.ru  
SYMBOLIKA_IMAP_PORT=993  
SYMBOLIKA_IMAP_SECURE=true  
SYMBOLIKA_IMAP_USER=start@symb62.ru  
SYMBOLIKA_IMAP_PASSWORD=пароль_почтового_ящика  
SYMBOLIKA_MAIL_ALLOWED_ALIASES=start@symb62.ru,manager1@symb62.ru,manager2@symb62.ru  
  
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru  
SYMBOLIKA_SMTP_PORT=465  
SYMBOLIKA_SMTP_SECURE=true  
SYMBOLIKA_SMTP_USER=start@symb62.ru  
SYMBOLIKA_SMTP_PASSWORD=пароль_почтового_ящика  
SYMBOLIKA_EMAIL_FROM=start@symb62.ru
```

После изменения переменных перезапустите Directus:

```
docker compose up -d --force-recreate directus
```

Папки и псевдонимы

В модуле `Почта` откройте настройки и для каждой строки укажите:

- точное имя папки на IMAP-сервере;
- псевдоним, от которого сотрудник отправляет ответы;
- сотрудника, которому доступна папка;
- признак **Общая**, если папку должны видеть все пользователи почты.

Администратор и управляющий видят все папки. Менеджер видит общие папки и папку, назначенную его карточке сотрудника.

Перед включением рабочего режима сначала проверьте синхронизацию на небольшом количестве писем и отправьте тестовое письмо на собственный внешний адрес.

Рабочая логика писем

- Почта автоматически обновляется каждые 10 секунд. Ручная кнопка обновления остается в верхней панели.
- Клиент и компания определяются по email отправителя. Если в теме есть номер вида **S0-00012**, система дополнительно пытается найти заказ.
- Ручные связи задаются кнопкой **Привязать** в открытом письме. Выбор заказа автоматически подставляет его клиента и компанию.
- В этом же окне письмо можно переложить в другую рабочую папку и назначить несколько тегов через запятую.
- Кнопка **Создать задачу** открывает выбор исполнителя, срока и приоритета. Созданная задача сохраняет связи письма с заказом, клиентом и компанией.
- Кнопка **Счет на оплату** создает важные задачи всем активным администраторам и управляющим. Повторная отправка одного счета защищена от дублей.
- Ссылка из созданной задачи возвращает в исходную почтовую переписку.
- Кнопка со стрелкой **Переслать** создает новое письмо с текстом исходного сообщения и перечнем его вложений.
- Собственная подпись настраивается через **Моя подпись**. Администратор и управляющий могут настроить подписи всей команды в настройках ящика.

Перемещение в рабочую папку изменяет организацию письма внутри учетной системы. Физическое перемещение исходного сообщения между папками IMAP требует отдельного хранения UID писем и в текущей версии не выполняется.

Подключение уведомлений клиентам: ВКонтakte, Telegram и Email

Полная актуальная инструкция по уведомлениям сотрудников находится в файле [Настройка уведомлений сотрудников](#). Этот файл описывает клиентский сценарий и будущий личный кабинет.

Эта инструкция описывает текущую реализацию уведомлений в системе «Символика», подключение внешних каналов и рекомендуемую схему будущего личного кабинета заказчика.

1. Что система отправляет сейчас

Автоматически отправляется только одно клиентское уведомление:

Заказ готов и находится в офисе.

Для отправки одновременно должны выполняться все условия:

1. статус заказа — **Готов** ;
2. способ получения — самовывоз из офиса;
3. офисный статус заказа — **В офисе** ;
4. все позиции заказа находятся в офисе;
5. у клиента или компании выбран основной канал уведомлений;
6. для выбранного канала указан получатель.

Сообщение содержит:

- номер заказа;
- позиции, количество, цену и сумму;
- общую сумму заказа;
- сколько оплачено;
- остаток к оплате;
- адрес и время работы офиса;
- сайт и ссылку на сообщество ВКонтakte, если они заполнены.

Повторно успешно отправленное уведомление по тому же заказу не отправляется. Результат и ошибка провайдера сохраняются в `symbolika_customer_notifications` .

Если в заказе выбраны и клиент, и компания, используется канал самого клиента. Настройки компании используются, только когда у клиента выбран вариант **Не отправлять** .

2. Где настраиваются уведомления

Общие данные офиса

Откройте:

Заполните:

- адрес офиса;
- режим работы;
- адрес сайта;
- ссылку на сообщество ВКонтакте.

Канал конкретного клиента

1. Откройте `Заказы → Клиенты и расчёты → Клиенты` или `Компании`.
2. Откройте карточку клиента/компании.
3. Найдите блок `Уведомление о готовом заказе в офисе`.
4. Выберите основной канал.
5. Укажите Email, Telegram chat ID или VK peer ID.

Изменение настроек клиента после перехода заказа в готовность запускает повторную проверку. Если уведомление по заказу уже имеет статус `sent`, второй раз оно не отправится.

3. Безопасность перед подключением

Токены ботов и пароли SMTP равнозначны паролям. Их нельзя:

- отправлять в чатах;
- вставлять в инструкции и скриншоты;
- хранить в Git;
- записывать непосредственно в исходный код.

В текущем `docker-compose.yml` значение `SYMBOLIKA_VK_TOKEN` задано непосредственно в файле. Перед рабочим подключением нужно:

1. отозвать текущий ключ сообщества ВКонтакте и выпустить новый;
2. заменить строку в `docker-compose.yml` на:

```
SYMBOLIKA_VK_TOKEN: "${SYMBOLIKA_VK_TOKEN:-}"
```

1. заменить жестко заданный локальный публичный адрес на:

```
SYMBOLIKA_PUBLIC_URL: "${SYMBOLIKA_PUBLIC_URL:-http://localhost:8057}"
```

1. хранить реальные ключи и публичный адрес только в файле `symbolika_directus_clean_install/.env`;
2. убедиться, что `.env` не попадает в Git (корневой `.gitignore` уже исключает его).

Пример `.env` без реальных секретов:

```
# Публичный адрес системы на VPS
SYMBOLIKA_PUBLIC_URL=https://account.example.ru

# ВКонтакте
SYMBOLIKA_VK_TOKEN=

# Telegram
SYMBOLIKA_TELEGRAM_BOT_TOKEN=

# Email / SMTP
SYMBOLIKA_SMTP_HOST=
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <mail@example.ru>"
```

После изменения `.env` контейнер надо пересоздать: обычный `docker restart` не перечитывает переменные окружения.

```
Set-Location symbolika_directus_clean_install
docker compose up -d --force-recreate directus
docker logs --tail 100 symbolika-directus
```

4. Подключение сообщества ВКонтакте

4.1. Подготовить сообщество

1. Откройте управление нужным сообществом ВКонтакте.
2. Включите сообщения сообщества.
3. В разделе работы с API создайте ключ доступа сообщества с правом отправки сообщений.
4. Скопируйте новый ключ в `SYMBOLIKA_VK_TOKEN` файла `.env`.
5. Не используйте пользовательский токен личной страницы.

Система вызывает метод VK API `messages.send`, передавая ключ сообщества, `peer_id`, `random_id` и текст сообщения.

Официальная документация:

- <https://dev.vk.com/ru/api/community-messages/getting-started>
- <https://dev.vk.com/ru/method/messages.send>

4.2. Подписать клиента

Сообщество не должно начинать диалог с человеком без его действия. Сначала клиент должен:

1. открыть сообщество;

2. нажать Сообщение ;
3. отправить любое сообщение, например Хочу получать уведомления о готовности заказа .

После этого нужно узнать числовой `peer_id` диалога. Пока входящий VK Callback API в системе не реализован, ID определяется вручную через список диалогов сообщества или метод `messages.getConversations` .

Пример проверки из PowerShell (токен не вставляется в команду и не сохраняется в истории):

```
$env:SYMBOLIKA_VK_TOKEN = Read-Host "VK token"
$vkResult = Invoke-RestMethod -Method Post `
  -Uri "https://api.vk.com/method/messages.getConversations" `
  -Body @{
    access_token = $env:SYMBOLIKA_VK_TOKEN
    v = "5.199"
    count = 20
  }
$vkResult.response.items | ForEach-Object {
  [PSCustomObject]@{
    peer_id = $_.conversation.peer.id
    message = $_.last_message.text
  }
}
```

В карточке клиента укажите:

Основной канал: ВКонтакте
VK peer ID: полученное числовое значение

Для личного диалога `peer_id` обычно совпадает с числовым ID пользователя. Не подставляйте короткое имя страницы вида `vk.com/username` .

5. Подключение Telegram-бота

5.1. Создать бота

1. Откройте официальный бот `@BotFather` .
2. Выполните команду `/newbot` .
3. Задайте имя и username, заканчивающийся на `bot` .
4. Сохраните полученный токен в `SYMBOLIKA_TELEGRAM_BOT_TOKEN` файла `.env` .
5. При компрометации токена немедленно перевыпустите его через BotFather.

Официальная инструкция Telegram: <<https://core.telegram.org/bots/tutorial>>.

5.2. Подписать клиента

Telegram-бот не может первым написать пользователю. Клиент должен:

1. открыть ссылку вида `https://t.me/ИМЯ_БОТА`;
2. нажать `Запустить` или отправить `/start`.

После этого получите `chat.id` через `getUpdates`:

```
$env:SYMBOLIKA_TELEGRAM_BOT_TOKEN = Read-Host "Telegram bot token"
$tgUpdates = Invoke-RestMethod `
    -Uri "https://api.telegram.org/bot$env:SYMBOLIKA_TELEGRAM_BOT_TOKEN/getUpdates"
$tgUpdates.result | ForEach-Object {
    [PSCustomObject]@{
        chat_id = $_.message.chat.id
        username = $_.message.chat.username
        name = ($_.message.chat.first_name, $_.message.chat.last_name -join " ").Trim()
        message = $_.message.text
    }
}
```

В карточке клиента укажите:

Основной канал: Telegram
Telegram chat ID: полученное числовое значение

Если у бота уже настроен webhook, `getUpdates` использовать одновременно нельзя. Проверить webhook можно методом `getWebhookInfo`. Текущая реализация «Символики» использует Telegram только для исходящей отправки и сама webhook не устанавливает.

Официальные материалы:

- `<https://core.telegram.org/bots>`
- `<https://core.telegram.org/bots/api#getupdates>`

6. Подключение Email

Нужен почтовый ящик или SMTP-сервис, разрешающий отpravку через SMTP. У провайдера получите:

- SMTP host;
- порт;
- логин;
- пароль приложения или SMTP-пароль;
- допустимый адрес отправителя.

Для порта `465`:

```
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
```

Для порта `587` обычно используется:


```
SYMBOLIKA_SMTP_PORT=587
SYMBOLIKA_SMTP_SECURE=false
```

Точный режим уточняйте у своего почтового провайдера. В Nodemailer порт 465 использует TLS сразу, а для 587 соединение обычно повышается до TLS после подключения: <<https://nodemailer.com/smtp>>.

Для рабочей доставляемости писем настройте у домена:

- SPF;
- DKIM;
- DMARC;
- адрес отправителя на том же подтвержденном домене.

В карточке клиента укажите:

```
Основной канал: Email
Email для уведомления: адрес заказчика
```

Сейчас письмо отправляется как обычный текст, без HTML-шаблона.

7. Как проверить отправку

Проверку лучше проводить на отдельном тестовом клиенте и тестовом заказе.

1. Настройте канал тестового клиента.
2. Создайте заказ со способом получения `Выдача в офисе`.
3. Добавьте хотя бы одну позицию.
4. Доведите заказ и позиции до статуса `Готов`.
5. Переведите все позиции в офисный статус `В офисе`.
6. Проверьте получение сообщения.
7. Проверьте журнал `symbolika_customer_notifications` и логи Directus.

```
docker logs --tail 150 symbolika-directus | Select-String -Pattern "customer
notification|Telegram|VK|SMTP|send failed"
```

Основные причины ошибок:

Ошибка	Что проверить
Не настроен Telegram-бот	<code>SYMBOLIKA_TELEGRAM_BOT_TOKEN</code> и пересоздание контейнера
Telegram chat not found	клиент запустил бота, chat ID указан без ошибок
Не настроена отправка ВКонтакте	ключ сообщества находится в окружении контейнера
VK отказал в отправке	сообщения сообщества включены, клиент начал диалог, указан правильный peer ID

Ошибка	Что проверить
Не настроена отправка email	host, user, password и from заполнены
SMTP authentication failed	пароль приложения, разрешение SMTP и параметры порта/TLS
Для выбранного канала не указан получатель	Email, chat ID, peer ID или телефон заполнены в карточке
Записи в журнале нет	заказ не удовлетворяет всем условиям готовности и нахождения в офисе
Статус <code>sent</code> , но повторного сообщения нет	защита от повторной отправки сработала штатно

Не выводите значения токенов командами `docker inspect`, `docker compose config` или в общие журналы/скриншоты.

8. Как клиент оформляет подписку сейчас

Самостоятельной подписки клиента пока нет. Рабочий процесс сейчас такой:

1. менеджер получает согласие клиента на выбранный канал;
2. клиент пишет сообществу ВКонтакте или запускает Telegram-бота;
3. менеджер получает peer ID/chat ID;
4. менеджер открывает карточку клиента или компании;
5. выбирает основной канал и вводит получателя;
6. при просьбе клиента отключить уведомления выбирает `Не отправлять`.

Согласие желательно фиксировать датой, источником и сотрудником, который его получил. Текущая модель хранит только выбранный канал и адрес получателя, но не хранит отдельный журнал согласий.

9. Рекомендуемый личный кабинет заказчика

Личный кабинет стоит делать отдельным внешним модулем. Не следует выдавать клиентам доступ в стандартную админку Directus или внутренние рабочие модули.

Что клиент должен видеть

- текущие и архивные заказы;
- позиции, количество, цены и публичные статусы;
- сроки и способ получения;
- сумму заказа, оплаты, остаток и переплату;
- общую сверку взаиморасчетов;
- контактные данные менеджера;
- публичные макеты/файлы, если они специально разрешены клиенту;
- настройки уведомлений и журнал отправленных сообщений.

Что клиент не должен видеть

- себестоимость и прибыль;
- контрагентов и закупочные цены;
- внутренние производственные комментарии;
- задачи сотрудников;
- внутреннюю ленту событий;
- служебные статусы и автоматизации;
- данные других клиентов и компаний.

Как оформить подписку безопасно

Email

1. Клиент вводит адрес.
2. Система отправляет ссылку подтверждения с одноразовым токеном.
3. Канал включается только после перехода по ссылке.

Telegram

1. Кабинет показывает кнопку Подключить Telegram .
2. Клиент открывает ссылку `https://t.me/ИМЯ_БОТА?start=ОДНОРАЗОВЫЙ_ТОКЕН` .
3. Telegram webhook получает `/start` и связывает `chat_id` с нужным клиентом.
4. В кабинете появляется статус Подключено .

ВКонтакте

1. Кабинет выдает одноразовый код или ссылку.
2. Клиент пишет этот код в сообщения сообщества.
3. VK Callback API принимает сообщение и связывает `peer_id` с клиентом.
4. В кабинете появляется статус Подключено .

Для VK и Telegram текущей исходящей отправки недостаточно: потребуется защищенный webhook для приема сообщений и таблица одноразовых токенов привязки.

Рекомендуемая модель данных

Не хранить подписку только в полях клиента. Добавить отдельную коллекцию, например `customer_notification_subscriptions` :

```
id
customer / customer_company
channel
recipient
status: pending / confirmed / disabled / failed
verification_token_hash
verification_expires_at
confirmed_at
disabled_at
consent_source
consent_ip
created_at
updated_at
```

Это позволит:

- подключить несколько каналов;
- выбрать один основной;
- хранить подтверждение согласия;
- безопасно отвязать старый Telegram/VK аккаунт;
- не перезаписывать контактные данные клиента;
- вести аудит подписок.

Доступ к кабинету

Рекомендуемый первый вариант — вход по одноразовой ссылке на Email. Для компаний позже можно добавить несколько пользователей и роли:

- владелец компании;
- бухгалтер;
- сотрудник, оформляющий заказы;
- только просмотр.

Ссылки должны быть короткоживущими и одноразовыми. Нельзя использовать постоянный `public_token` позиции как авторизацию ко всему кабинету.

10. Рекомендуемый порядок реализации

Этап 1 — довести текущие уведомления

- вынести все секреты в `.env` ;
- добавить кнопку `Отправить тестовое уведомление` ;
- добавить понятный журнал отправок в рабочий модуль;
- добавить кнопку повторной отправки для статуса `failed` ;
- добавить проверку доступности канала до сохранения.

Этап 2 — страница самостоятельной подписки

- отдельная публичная страница без полного личного кабинета;
- подтверждение Email;
- Telegram deep link и webhook;
- VK Callback API и одноразовый код;
- отключение подписки.

Этап 3 — полноценный кабинет заказчика

- заказы и позиции;
- финансы и сверка;
- файлы;
- менеджер и контакты;
- настройки уведомлений;
- доступ нескольких сотрудников компании.

Такой порядок быстрее даст безопасную самостоятельную подписку и не заставит сразу строить весь клиентский портал.

Настройка уведомлений «Символики»

Краткая карта всех токенов, правила хранения и обязательная ротация перед релизом: [Токены и секреты](#).

Инструкция относится к текущей реализации системы и описывает два независимых контура:

1. уведомления сотрудникам о заказах, позициях, задачах, производстве, закупках, письмах и финансах;
2. автоматическое уведомление клиента «Заказ готов и находится в офисе».

SMS в текущей версии не реализованы. Центр уведомлений внутри системы работает без внешних провайдеров. Push, Email, ВКонтакте и Telegram требуют отдельной настройки.

1. Что подготовить

Понадобятся:

- публичный HTTPS-адрес системы, например `https://account.example.ru`;
- пара VAPID-ключей для браузерных Push;
- ключ доступа сообщества ВКонтакте;
- Telegram-бот и его токен;
- параметры SMTP почты `start@symb62.ru` на REG.RU;
- доступ к файлу `.env` рядом с `docker-compose.yml` на сервере.

Не пересылайте токены и пароли в чатах, не добавляйте их в Git и не вставляйте в скриншоты.

2. Обязательная подготовка `docker-compose.yml`

В текущем файле исторически присутствуют встроенные значения Push и VK. Перед рабочим запуском:

1. перевыпустите ключ сообщества ВКонтакте;
2. выпустите новую пару VAPID-ключей;
3. замените значения соответствующих строк в `environment` сервиса Directus на ссылки на `.env`:

```
SYMBOLIKA_PUBLIC_URL: "${SYMBOLIKA_PUBLIC_URL:-http://localhost:8057}"
SYMBOLIKA_PUSH_PUBLIC_KEY: "${SYMBOLIKA_PUSH_PUBLIC_KEY:-}"
SYMBOLIKA_PUSH_PRIVATE_KEY: "${SYMBOLIKA_PUSH_PRIVATE_KEY:-}"
SYMBOLIKA_PUSH_SUBJECT: "${SYMBOLIKA_PUSH_SUBJECT:-mailto:start@symb62.ru}"
SYMBOLIKA_VK_TOKEN: "${SYMBOLIKA_VK_TOKEN:-}"
SYMBOLIKA_VK_API_VERSION: "${SYMBOLIKA_VK_API_VERSION:-5.199}"
SYMBOLIKA_VK_PRODUCTION_PEER_ID: "${SYMBOLIKA_VK_PRODUCTION_PEER_ID:-}"
SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID: "${SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID:-}"
```

Telegram и SMTP уже должны использовать аналогичные ссылки на `.env`. Не публикуйте результат команды `docker compose config`: он может содержать подставленные секреты.

3. Заполнить `.env` на сервере

Откройте каталог установки и создайте `.env`, если его ещё нет:

```
cd /path/to/symbolika_directus_clean_install
cp -n .env.example .env
nano .env
```

Заполните параметры:

```
# Публичный адрес системы. На сервере обязательно HTTPS.
SYMBOLIKA_PUBLIC_URL=https://YOUR_DOMAIN

# Push
SYMBOLIKA_PUSH_PUBLIC_KEY=
SYMBOLIKA_PUSH_PRIVATE_KEY=
SYMBOLIKA_PUSH_SUBJECT=mailto:start@symb62.ru

# ВКонтакте
SYMBOLIKA_VK_TOKEN=
SYMBOLIKA_VK_API_VERSION=5.199
SYMBOLIKA_VK_PRODUCTION_PEER_ID=
SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID=

# Telegram
SYMBOLIKA_TELEGRAM_BOT_TOKEN=

# Отправка Email через REG.RU
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=start@symb62.ru
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <start@symb62.ru>"
```

Параметры IMAP встроенного почтового клиента настраиваются отдельно по инструкции [Настройка встроенной почты](#). Для уведомлений достаточно SMTP-параметров выше.

4. Настроить браузерные Push

4.1. Сгенерировать VAPID-ключи

Ключи создаются один раз и затем постоянно используются сервером. Например:

```
npx web-push generate-vapid-keys --json
```

Скопируйте `publicKey` в `SYMBOLIKA_PUSH_PUBLIC_KEY`, а `privateKey` — в `SYMBOLIKA_PUSH_PRIVATE_KEY`. Закройте терминал после копирования и не сохраняйте его вывод в общий журнал.

`SYMBOLIKA_PUSH_SUBJECT` должен быть адресом вида `mailto:start@symb62.ru` либо публичным HTTPS URL.

4.2. Подключить устройство сотрудника

После запуска системы каждый сотрудник выполняет это на каждом нужном компьютере или телефоне:

1. войти в систему;
2. открыть иконку колокольчика `Центр уведомлений`;
3. открыть вкладку `Каналы доставки`;
4. включить `Push` в браузере;
5. нажать `Подключить это устройство`;
6. разрешить уведомления в диалоге браузера;
7. нажать `Сохранить настройки`.

На сервере Push требуют HTTPS. `localhost` является допустимым исключением только для локальной проверки. Если пользователь запретил уведомления, разрешение нужно вернуть в настройках сайта самого браузера.

5. Настроить Telegram

5.1. Создать бота

1. Откройте официального бота `@BotFather`.
2. Выполните `/newbot`.
3. Укажите название и username, заканчивающийся на `bot`.
4. Сохраните полученный токен в `SYMBOLIKA_TELEGRAM_BOT_TOKEN`.

Официальная документация: <https://core.telegram.org/bots/tutorial>.

5.2. Получить `chat_id`

Бот не может первым начать личный диалог. Сотрудник или клиент должен открыть бота и нажать `Запустить` либо отправить `/start`.

Для разовой настройки получите `chat.id`:


```

$token = Read-Host "Telegram bot token"
$updates = Invoke-RestMethod "https://api.telegram.org/bot$token/getUpdates"
$updates.result | ForEach-Object {
    [PSCustomObject]@{
        chat_id = $_.message.chat.id
        username = $_.message.chat.username
        name = ($_.message.chat.first_name, $_.message.chat.last_name -join " ").Trim()
        text = $_.message.text
    }
}
$token = $null

```

Если у бота установлен webhook, `getUpdates` одновременно не работает. Telegram поддерживает либо webhook, либо long polling: <<https://core.telegram.org/bots/faq#getting-updates>>.

Для сотрудника `chat_id` вводится в Центр уведомлений → Каналы доставки → Telegram. Для клиента — в карточке клиента или компании.

6. Настроить ВКонтакте

6.1. Подготовить сообщество

1. В управлении сообществом включите сообщения.
2. В разделе API создайте ключ доступа сообщества с правом отправки сообщений.
3. Запишите новый ключ в `SYMBOLIKA_VK_TOKEN`.
4. Не используйте токен личной страницы.

Система отправляет сообщения методом `messages.send`, используя `peer_id` получателя.

6.2. Получить `peer_id`

Сотрудник или клиент сначала должен написать сообществу любое сообщение. Затем `peer_id` можно получить из диалогов сообщества:

```

$token = Read-Host "VK community token"
$result = Invoke-RestMethod -Method Post `
    -Uri "https://api.vk.com/method/messages.getConversations" `
    -Body @{ access_token = $token; v = "5.199"; count = 50 }
$result.response.items | ForEach-Object {
    [PSCustomObject]@{
        peer_id = $_.conversation.peer.id
        text = $_.last_message.text
    }
}
$token = $null

```

Для сотрудника `peer_id` вводится в Центр уведомлений → Каналы доставки → ВКонтакте . Для клиента — в карточке клиента или компании. Не вводите короткое имя страницы вида `vk.com/username` .

7. Настроить отправку Email через REG.RU

Для почтового хостинга REG.RU текущая конфигурация использует:

```
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=start@symb62.ru
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <start@symb62.ru>"
```

REG.RU указывает `mail.hosting.reg.ru` как сервер исходящей почты для защищённого соединения: <https://help.reg.ru/support/hosting/nastroyka-pochty-regru/nastroyka-pochty-i-pochtovykh-kliientovv/nastroyka-pochtovykh-kliientovv>.

Для нормальной доставки проверьте DNS домена:

- SPF разрешает серверу REG.RU отправлять письма от имени домена;
- DKIM включён в панели почтового хостинга;
- DMARC добавлен хотя бы в режиме мониторинга.

Сотрудник включает Email в своём центре уведомлений и указывает адрес. Если адрес не указан, сервер может использовать Email учётной записи Directus.

8. Применить настройки на сервере

Обычный `docker restart` не перечитывает `.env` . Контейнер Directus нужно пересоздать:

```
cd /path/to/symbolika_directus_clean_install
docker compose up -d --force-recreate directus
docker logs --tail 120 symbolika-directus
```

Проверьте наличие настроек, не выводя их значения:

```
docker exec symbolika-directus node -e '
const names = [
  "SYMBOLIKA_PUBLIC_URL",
  "SYMBOLIKA_PUSH_PUBLIC_KEY",
  "SYMBOLIKA_PUSH_PRIVATE_KEY",
  "SYMBOLIKA_VK_TOKEN",
  "SYMBOLIKA_TELEGRAM_BOT_TOKEN",
  "SYMBOLIKA_SMTP_HOST",
  "SYMBOLIKA_SMTP_USER",
  "SYMBOLIKA_SMTP_PASSWORD",
  "SYMBOLIKA_EMAIL_FROM"
];
for (const name of names) console.log(name + ": " + (process.env[name] ? "OK" : "НЕ
ЗАПОЛНЕНО"));
```

9. Персональная настройка сотрудника

Каждый сотрудник открывает **Центр уведомлений** → **Каналы доставки** и выбирает события:

- статусы заказов;
- позиции и макеты;
- новые задачи;
- изменения задач;
- производство;
- закупки;
- новые письма;
- финансы.

Критические системные события не отключаются. Для каждого внешнего канала нужно включить переключатель и указать получателя. После изменений нажать **Сохранить настройки**.

Уведомление внутри центра системы является основным. Внешние каналы доставляют его копию и ссылку на связанный заказ, позицию или задачу.

10. Настройка уведомления клиенту

Сейчас клиенту автоматически отправляется только сообщение:

Заказ готов и находится в офисе.

Другие статусы клиенту не отправляются.

10.1. Заполнить общие данные

Администратор открывает:

Нужно заполнить:

- адрес офиса;
- режим работы;
- сайт;
- ссылку на группу ВКонтакте.

10.2. Выбрать канал клиента

1. Откройте **Заказы → Клиенты и расчёты → Клиенты** или **Компании**.
2. Откройте карточку.
3. В блоке уведомления о готовом заказе выберите **Email**, **Telegram**, **ВКонтакте** или **Не отправлять**.
4. Укажите **Email**, **chat_id** или **peer_id**.

Если в заказе выбраны клиент и компания, сначала используется канал клиента. Настройки компании применяются, если у клиента выбран вариант **Не отправлять**.

10.3. Условия автоматической отправки

Сообщение уйдёт только когда одновременно выполнено всё:

1. заказ имеет статус **Готов**;
2. способ получения — **выдача в офисе**;
3. офисный статус заказа — **В офисе**;
4. все позиции находятся в офисе;
5. у клиента или компании выбран канал;
6. заполнен получатель выбранного канала.

В сообщение включаются позиции, количество, цены, сумма, оплачено, остаток, адрес и режим работы офиса. Успешно отправленное сообщение повторно по тому же заказу не отправляется.

11. Пошаговая проверка

Проверка сотрудника

1. Создайте тестового сотрудника или используйте свой аккаунт.
2. Подключите один канал в центре уведомлений.
3. Включите тему **Новые задачи**.
4. Назначьте этому сотруднику новую задачу с другого аккаунта.
5. Проверьте запись в центре и внешнее сообщение.
6. В центре возле внешнего канала должен появиться статус **доставлено** или **ошибка**.

Проверка клиента

1. Создайте тестового клиента со своим **Email**, **chat_id** или **peer_id**.

2. Создайте заказ с выдачей в офисе.
3. Добавьте позицию и доведите её до готовности.
4. Установите офисный статус `В офисе`.
5. Проверьте сообщение и журнал клиентских уведомлений.

Проверка логов

```
docker logs --tail 200 symbolika-directus 2>&1 | grep -Ei
"notification|telegram|vk|smtp|push|send failed"
```

В логи не должны попадать сами токены и пароли.

12. Частые ошибки

Симптом	Что проверить
Push сообщает, что ключ не настроен	VAPID public/private присутствуют в окружении пересозданного контейнера
Браузер не спрашивает разрешение	HTTPS, разрешения сайта, поддержка Service Worker и Push API
Telegram <code>chat not found</code>	пользователь запустил бота; введён правильный <code>chat_id</code>
<code>getUpdates</code> не возвращает сообщения	у бота установлен webhook либо пользователь ещё не написал боту
VK не отправляет	сообщения сообщества включены; клиент начал диалог; токен сообщества и <code>peer_id</code> верны
SMTP authentication failed	логин, пароль ящика, порт 465 и защищённое соединение
В центре есть уведомление, внешнего сообщения нет	канал выключен, не указан получатель либо провайдер вернул ошибку
Клиентское уведомление не создано	не выполнено одно из условий готовности и нахождения в офисе
Уведомление имеет статус <code>sent</code> , но повторно не приходит	штатно сработала защита от дублей

13. Рекомендуемый порядок включения

1. Перевыпустить и вынести секреты из `docker-compose.yml` в `.env`.
2. Настроить `SYMBOLIKA_PUBLIC_URL` и HTTPS.
3. Подключить SMTP и проверить Email.
4. Настроить Telegram-бота и тестовый `chat_id`.
5. Настроить сообщество VK и тестовый `peer_id`.
6. Выпустить VAPID-ключи и подключить Push на одном устройстве.
7. Провести тест назначения задачи сотруднику.

8. Провести тест готового клиентского заказа в офисе.

9. Подключать сотрудников и клиентов постепенно, проверяя журнал доставки.

VAPID-ключи следует создать один раз и хранить постоянно; это рекомендует используемая библиотека `web-push` : <https://github.com/web-push-libs/web-push>.

Яндекс Диск: хранение макетов

Что реализовано

В форме нового заказа, форме добавления позиции и сохраненной карточке позиции есть загрузка макета. Она открывает отдельное окно, куда файл можно перетащить или выбрать через проводник. Перед отправкой показываются имя и размер файла. Для новой позиции файл сначала прикрепляется к черновику браузера, а сразу после создания позиции передается через Directus на Яндекс Диск. В `orders_items.url` сохраняется публичная ссылка.

Ручной ввод адреса макета в формах заказа и позиции отключен. В сохраненной карточке ссылка показывается только для просмотра и открытия файла; заменить макет можно через повторную загрузку.

Файлы раскладываются по структуре:

```
app:/Заказы/<номер заказа>/Позиция-<id>/<дата-время>-<имя файла>
```

Повторная загрузка не перезаписывает старый макет: создается новый файл, а позиция начинает ссылаться на новую версию.

Переменные окружения

Скопировать значения из `.env.example` в локальный `.env`:

```
SYMBOLIKA_YANDEX_DISK_TOKEN=  
SYMBOLIKA_YANDEX_DISK_ROOT=app:/Заказы  
SYMBOLIKA_YANDEX_DISK_PUBLISH_FILES=true
```

Токен нельзя добавлять в `.env.example`, Git или клиентский JavaScript. Приложению Яндекса достаточно прав `cloud_api:disk.app_folder` и `cloud_api:disk.info`.

После изменения `.env` пересоздать Directus:

```
docker compose -f symbolika_directus_clean_install\docker-compose.yml up -d --force-recreate  
directus
```

Проверка

После авторизации защищенный endpoint должен вернуть `configured: true` и `connected: true`:

```
GET /symbolika-yandex-disk/status
```

Загрузка выполняется через:

```
POST /symbolika-yandex-disk/orders-items/<id>/upload
```

Endpoint принимает тело файла без multipart-обертки и заголовки `X-File-Name`, `X-File-Size`, `Content-Type`. Максимальный размер через интерфейс — 2 ГБ. Для более крупных файлов остается ручная вставка ссылки.

Развертывание на VPS

На сервере токен заносится только в `.env` рядом с `docker-compose.yml`. После получения кода и базы нужно применить `setup/create-work-views.sql` и пересоздать контейнер Directus. Сам токен в дампе PostgreSQL не попадает.

Релиз «Символика» от 10.08.2026

Статус

Релизный кандидат подготовлен для переноса на Linux/VPS.

Проверено

- синтаксис 26 JavaScript-файлов;
- разбор `docker-compose.yml` с внешним `.env`;
- применение `setup/create-work-views.sql` с `ON_ERROR_STOP=1`;
- загрузка всех расширений Directus;
- `/server/health` — HTTP 200, `status: ok`;
- финальный регресс: 28 проверок, 28 успешно;
- очистка созданных QA-данных после теста;
- удаление одноразового QA-токена;
- secret scan рабочего дерева;
- PostgreSQL custom dump создан и проверен через `pg_restore --list`.

Покрытие финального регресса

- семь пользовательских ролей и авторизация;
- изоляция данных менеджеров;
- создание клиента, компании, заказа и позиции;
- запуск без обязательной цены;
- суммы, частичная оплата и распределение;
- маршрутизация и производственные статусы;
- офис и выдача;
- дизайнерская задача, обсуждение, чек-лист и результат;
- складской минимум, закупочный пакет и задача;
- права управляющего и менеджера на здоровье автоматизаций;
- публичная ссылка позиции.

Безопасность релиза

- секреты удалены из `docker-compose.yml` и вынесены в `.env`;
- известные тестовые пароли удалены из чистого импорта;
- Git remote очищен от встроенного персонального токена;
- перед боевым запуском обязательна ротация старых GitHub, VK и VAPID-секретов, а также тестовых паролей.

Файлы установки

- Загрузка релиза на сервер
- Первоначальная настройка сервера
- Токены и секреты
- Настройка уведомлений сотрудников
- Чек-лист релиза

Релизный dump хранится локально в `backups/` и не входит в Git.

Замечания

- Directus зафиксирован на версии `11.17.0` ; обновление до 12.x не выполнять одновременно с первым боевым развертыванием.
- В базе присутствуют исторические backup-таблицы metadata без primary key. Directus предупреждает о них, но рабочие расширения и API загружаются успешно. Их чистку следует выполнять отдельной задачей после резервной копии.
- Сначала развернуть текущую проверенную версию, затем отдельно планировать обновление Directus и рефакторинг большого рабочего модуля.

Релиз «Символика» от 11.08.2026

Статус

Релиз подготовлен, проверен и отправлен в ветку `dev-v1`.

- релизный коммит: `df2fd4f3171fdf17644eba5e7ea72afc209aa79e` ;
- сообщение коммита: `Release unified archives and order navigation` ;
- локальный `HEAD` и `origin/dev-v1` совпадают;
- Directus: `11.17.0` ;
- рабочее дерево после выпуска было чистым.

Последние изменения интерфейса

- отдельные страницы `Архив заказов` , `Архив позиций` и `Архив производства` убраны из рабочего меню;
- в заказах и очередях участков используется переключатель `Активные / Архив / Все` ;
- в заказах архив работает отдельно для режима `По заказам` и `По позициям` ;
- фильтр по конкретному статусу ищет записи независимо от выбранного режима архива;
- клиенты, компании, сверки, клиентские операции и подарочные сертификаты перенесены внутрь модуля `Заказы` в группу `Клиенты и расчёты` ;
- отдельные главные модули `Клиенты` и `Финансы` скрыты;
- переходы из почты, профиля и уведомлений на клиента или компанию ведут в модуль `Заказы` ;
- старые сохранённые ссылки на страницы архивов автоматически переводятся в новый архивный режим.

Проверено перед выпуском

- синтаксис изменённых JavaScript-файлов через `node --check` ;
- `git diff --check` ;
- применение `setup/create-work-views.sql` с `ON_ERROR_STOP=1` ;
- запуск Directus и отсутствие ошибок расширений в логах;
- финальный Playwright-регресс всех рабочих ролей;
- 20 экранов, включая заказы, производство, управление, закупки, почту и профиль;
- наличие новых архивных переключателей;
- отсутствие отдельных иконок клиентов и финансов;
- очистка временных QA-пользователей и одноразового токена;
- отсутствие `.env` , дампов, базы и runtime-файлов в коммите.

Результат финального Playwright-прогона: `passed: true` , ошибок: `0` .

Резервная копия

Перед выпуском создан PostgreSQL custom dump:

```
backups/symbolika-release-20260811-000535.dump
```

- размер: 962222 байта;
- формат проверен через `pg_restore --list`;
- в дампе обнаружено 1037 объектов;
- дампы хранятся локально и не входят в Git.

Порядок развёртывания

1. Получить ветку `dev-v1` и убедиться, что доступен коммит `df2fd4f`.
2. Создать защищённый серверный `.env` по инструкции `Токены и секреты.md`.
3. Восстановить проверенный custom dump либо использовать существующую боевую базу после отдельного бэкапа.
4. Запустить контейнеры.
5. Применить `setup/create-work-views.sql`.
6. Перезапустить Directus и проверить логи.
7. Выполнить приёмочный сценарий под каждой используемой ролью.

Подробные команды находятся в файлах `Загрузка релиза на сервер.md` и `Первоначальная настройка сервера.md`.

Первоначальная настройка сервера

1. Системные требования

- Linux x86_64;
- Docker Engine и `docker compose`;
- не менее 4 ГБ RAM, рекомендуется 8 ГБ;
- свободное место для базы, резервных копий и `uploads`;
- домен с HTTPS;
- исходящий доступ к GitHub, REG.RU, Яндекс, VK и Telegram.

Открывать PostgreSQL наружу не нужно. Порт 8057 лучше закрыть firewall и отдавать только через HTTPS reverse proxy.

2. Создать `.env`

```
cd /opt/symbolika/directus_symb_by_vps/symbolika_directus_clean_install
cp .env.example .env
chmod 600 .env
nano .env
```

Обязательные значения и способы получения описаны в инструкции [Токены и секреты](#).

Минимум для запуска:

```
DIRECTUS_KEY=<случайная строка>
DIRECTUS_SECRET=<случайная строка>
ADMIN_EMAIL=<почта администратора>
ADMIN_PASSWORD=<сильный пароль>
POSTGRES_PASSWORD=<сильный пароль базы>
SYMBOLIKA_PUBLIC_URL=https://your-domain.example
```

3. Установить зависимости расширений

Внешние npm-зависимости нужны двум endpoint-модулям. Установите их воспроизводимо из lock-файлов:

```
docker run --rm -v "$PWD/extensions/symbolika-calculations:/app" -w /app node:20-alpine npm ci --omit=dev
docker run --rm -v "$PWD/extensions/symbolika-mail:/app" -w /app node:20-alpine npm ci --omit=dev
```

Повторяйте команды после изменения соответствующего `package-lock.json`.

4. Восстановить релизную базу

Рекомендуемый способ первого боевого запуска — восстановление проверенного release dump.

Запустите только базу:

```
docker compose up -d database
docker compose ps
```

Скопируйте дамп и восстановите:

```
docker cp /opt/symbolika/directus-release.dump symbolika-db:/tmp/directus.dump
docker exec symbolika-db pg_restore -U directus -d directus --clean --if-exists --no-owner
/tmp/directus.dump
```

Если база пустая, предупреждения об отсутствующих объектах при `--clean --if-exists` допустимы; итоговая команда должна завершиться без фатальной ошибки.

5. Запустить Directus

```
docker compose up -d directus
docker compose ps
docker logs --tail 150 symbolika-directus
```

Оба контейнера должны иметь состояние `Up`, база — `healthy`. В логах не должно быть циклических перезапусков, ошибок загрузки extensions или подключения к PostgreSQL.

6. Применить SQL текущей версии

После восстановления дампа примените идемпотентный SQL-слой из текущего кода:

```
docker exec -i symbolika-db psql -v ON_ERROR_STOP=1 -U directus -d directus < setup/create-
work-views.sql
docker restart symbolika-directus
docker logs --tail 150 symbolika-directus
```

Это синхронизирует права, роли, представления, триггеры и metadata с релизом.

7. Настроить домен и HTTPS

В ISPmanager создайте WWW-домен, выпустите Let's Encrypt и настройте проху на:

```
http://127.0.0.1:8057
```

Обязательно разрешите WebSocket и передавайте заголовки `Host`, `X-Forwarded-Proto`, `X-Forwarded-For`. После появления HTTPS значение `SYMBOLIKA_PUBLIC_URL` должно точно совпадать с внешним

адресом без `/admin`.

После изменения `.env` пересоздайте контейнер:

```
docker compose up -d --force-recreate directus
```

8. Первый вход

1. Откройте `https://your-domain.example/admin/`.
2. Войдите под `ADMIN_EMAIL` и `ADMIN_PASSWORD`.
3. Сразу смените временный пароль, если он использовался.
4. Проверьте сотрудников, пользователей, роли и связь `employee` → `directus_user`.
5. Проверьте роли администратора, управляющего, менеджера, производства, шелкографии, офиса и дизайнера.
6. Войдите тестовым пользователем каждой роли.

9. Подключить интеграции

Подключайте по одной, каждый раз пересоздавая Directus и проверяя результат:

1. Яндекс Диск.
2. SMTP и IMAP.
3. Browser Push.
4. Telegram.
5. VK.
6. SMS.ru, только если канал нужен.

Точные переменные: **Токены и секреты**. Полная проверка: **Настройка уведомлений сотрудников**.

10. Приемочная проверка

- создать клиента и заказ;
- добавить позицию и загрузить макет;
- отправить заказ в работу;
- проверить очередь участка;
- создать дизайнерскую задачу и завершить с результатом;
- создать закупочную заявку и проверить пакет/задачу;
- поставить готовность, принять в офис, добавить оплату и выдать;
- проверить уведомления, почту, события и контроль автоматизаций;
- проверить светлую и темную тему, смартфон и планшет.

11. Резервное копирование

Настройте ежедневный `pg_dump -Fc`, отдельный архив `uploads` и хранение вне сервера. Не считайте резервной копией сам каталог PostgreSQL.

Проверяйте каждый дамп через `pg_restore --list`, а восстановление — на отдельной тестовой базе.

Рабочие инструкции системы «Символика»

Инструкции разделены по ролям. Каждый файл можно передать сотруднику отдельно.

- Менеджер
- Производство и шелкография
- Дизайнер
- Управляющий
- Администратор

Документы описывают фактически реализованную логику рабочих модулей. Стандартные коллекции Directus являются техническим уровнем и не предназначены для повседневной работы сотрудников.

Общие правила для всех ролей:

- работать через разделы системы в левом меню;
- не исправлять вручную расчетные и служебные поля;
- пользоваться Центром уведомлений для рабочих и системных сообщений;
- в Личном кабинете поддерживать актуальные контакты, аватар, тему и каналы уведомлений;
- при ошибке использовать кнопку Сообщить об ошибке : система приложит пользователя, страницу и открытую сущность;
- важное действие считать завершенным только после успешного сохранения и сообщения системы.

Токены, пароли и переменные окружения

Все секреты записываются только в серверный файл:

```
symbolika_directus_clean_install/.env
```

Права файла: `chmod 600 .env`. Не вставляйте секреты в `docker-compose.yml`, `.env.example`, Git, документацию, скриншоты или чат.

1. Обязательные системные секреты

Переменная	Что вставить	Где получить
<code>DIRECTUS_KEY</code>	случайную строку 32+ байта	<code>openssl rand -hex 32</code>
<code>DIRECTUS_SECRET</code>	другую случайную строку 32+ байта	<code>openssl rand -hex 32</code>
<code>ADMIN_EMAIL</code>	email первого администратора	выбрать самостоятельно
<code>ADMIN_PASSWORD</code>	уникальный сильный пароль	менеджер паролей
<code>POSTGRES_PASSWORD</code>	уникальный пароль базы	менеджер паролей
<code>SYMBOLIKA_PUBLIC_URL</code>	внешний HTTPS URL без <code>/admin</code>	ваш домен
<code>DIRECTUS_CORS_ORIGIN</code>	домен системы либо <code>true</code> на первом запуске	настройка администратора

`DIRECTUS_KEY` и `DIRECTUS_SECRET` после запуска не менять без плана миграции сессий и токенов.

2. Яндекс Диск

Переменная	Значение
<code>SYMBOLIKA_YANDEX_DISK_TOKEN</code>	OAuth-токен приложения Яндекса
<code>SYMBOLIKA_YANDEX_DISK_ROOT</code>	<code>app:/Заказы</code>
<code>SYMBOLIKA_YANDEX_DISK_PUBLISH_FILES</code>	<code>true</code>
<code>SYMBOLIKA_YANDEX_DISK_DELETE_REPLACED</code>	<code>true</code>

Приложению Яндекса нужны только права `cloud_api:disk,app_folder` и `cloud_api:disk.info`. Полный доступ ко всему Диску не выдавать.

Проверка после перезапуска:

```
GET https://YOUR_DOMAIN/symbolika-yandex-disk/status
```

Ответ авторизованному администратору: `configured: true`, `connected: true`.

3. Browser Push

Переменная	Значение
<code>SYMBOLIKA_PUSH_PUBLIC_KEY</code>	публичный VAPID key
<code>SYMBOLIKA_PUSH_PRIVATE_KEY</code>	приватный VAPID key
<code>SYMBOLIKA_PUSH_SUBJECT</code>	<code>mailto:start@symb62.ru</code>

Сгенерировать одну пару:

```
docker run --rm node:20-alpine sh -c "npx --yes web-push generate-vapid-keys"
```

Push работает только через HTTPS, кроме localhost. Приватный ключ не показывать пользователям.

4. ВКонтакте

Переменная	Значение
<code>SYMBOLIKA_VK_TOKEN</code>	токен сообщества с правом сообщений
<code>SYMBOLIKA_VK_API_VERSION</code>	5.199
<code>SYMBOLIKA_VK_PRODUCTION_PEER_ID</code>	peer_id чата/получателя производства
<code>SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID</code>	peer_id шелкографии

Токен создается в управлении сообществом VK. Используйте токен сообщества, не личный пользовательский токен. Для персональных уведомлений сотрудник указывает свой идентификатор в настройках центра уведомлений.

После создания нового токена старый отзовите. Если токен когда-либо попадал в Git или URL, он считается скомпрометированным.

5. Telegram

Переменная	Значение
<code>SYMBOLIKA_TELEGRAM_BOT_TOKEN</code>	токен от @BotFather

Каждый сотрудник/клиент сначала пишет боту `/start`, после чего в карточке или настройках указывается его `chat_id`. Токен один на систему, `chat_id` индивидуальны.

6. SMTP для отправки почты

Для REG.RU:

```
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=start@symb62.ru
SYMBOLIKA_SMTP_PASSWORD=<пароль основного ящика>
SYMBOLIKA_EMAIL_FROM="Символика <start@symb62.ru>"
```

Используйте пароль почтового ящика, не пароль ISPmanager.

7. IMAP для встроенной почты

```
SYMBOLIKA_MAIL_MODE=imap
SYMBOLIKA_IMAP_HOST=mail.hosting.reg.ru
SYMBOLIKA_IMAP_PORT=993
SYMBOLIKA_IMAP_SECURE=true
SYMBOLIKA_IMAP_USER=start@symb62.ru
SYMBOLIKA_IMAP_PASSWORD=<пароль основного ящика>
SYMBOLIKA_MAIL_ALLOWED_ALIASES=start@symb62.ru,alias1@symb62.ru,alias2@symb62.ru
```

Для безопасной проверки оставьте `SYMBOLIKA_MAIL_MODE=mock`. После проверки папок, подписей и прав переключите на `imap`.

Псевдонимы перечисляются через запятую без пробелов. Они должны реально поддерживать отправку на почтовом хостинге.

8. SMS.ru — необязательно

```
SYMBOLIKA_SMS_RU_API_ID=<api_id из кабинета SMS.ru>
SYMBOLIKA_SMS_RU_FROM=<согласованное имя отправителя>
```

Не заполняйте эти поля, если SMS-канал не используется. Имя отправителя должно быть предварительно согласовано у провайдера.

9. Где хранятся идентификаторы получателей

Системный `.env` содержит ключи провайдеров. Конкретные адресаты задаются в интерфейсе:

- сотрудник — Центр уведомлений → Настройки : email, VK, Telegram, push;
- клиент/компания — карточка: основной канал и адрес/идентификатор;
- производство и шелкография — общие `peer_id` в `.env`;
- почтовые псевдонимы и папки — настройки модуля Почта.

10. Применение изменений

```
docker compose up -d --force-recreate directus
docker logs --tail 150 symbolika-directus
```

Проверяйте один канал за раз. Ошибки доставки смотрите в [Управление → Контроль автоматизаций](#) и в журнале центра уведомлений.

11. Обязательная ротация перед боевым запуском

Перед релизом замените:

- старые пароли администратора и PostgreSQL;
- `DIRECTUS_KEY` и `DIRECTUS_SECRET`, если использовались тестовые;
- VK-токен, если он когда-либо был записан в репозиторий;
- GitHub PAT, если он когда-либо находился в URL remote;
- SMTP/IMAP пароль, если передавался небезопасным способом;
- тестовые VAPID-ключи, если их приватная часть была опубликована.

Чек-лист релиза

До отправки

- [] Все секреты вынесены в `.env`.
- [] В Git нет `.env`, базы, dumps, uploads, node_modules и временных скриншотов.
- [] Git remote не содержит токенов.
- [] GitHub PAT и опубликованные provider tokens отозваны.
- [] JS проходит `node --check`.
- [] SQL применяется с `ON_ERROR_STOP=1`.
- [] Directus стартует без ошибок extensions.
- [] Выполнен Playwright smoke/regression по ролям.
- [] Создан `pg_dump -Fc` и проверен через `pg_restore --list`.
- [] Подготовлен архив uploads, если локальные файлы нужны на сервере.
- [] Shell-скрипты имеют LF.

На сервере

- [] `.env` имеет права 600.
- [] PostgreSQL не опубликован наружу.
- [] Directus доступен через HTTPS.
- [] WebSocket работает через reverse proxy.
- [] Выполнен `restore release dump`.
- [] Применен `setup/create-work-views.sql`.
- [] Установлены npm-зависимости calculations и mail.
- [] Проверены все роли и связи пользователей с сотрудниками.
- [] Подключены и отдельно протестированы Яндекс Диск, SMTP/IMAP, Push, Telegram и VK.
- [] Настроен ежедневный off-server backup.
- [] Выполнена приемочная бизнес-цепочка от заказа до выдачи.

Чистая установка Directus «Символика»

Этот пакет поднимает новый Directus рядом со старым, на порту `8057`.

Старый Directus не трогаем.

1. Загрузить архив на сервер

```
scp symbolika_directus_clean_install.zip root@IP_СЕРВЕРА:/root/
```

2. Распаковать

```
cd /root
unzip symbolika_directus_clean_install.zip
cd symbolika_directus_clean_install
```

3. Запустить новый чистый Directus

Сначала создайте защищенный `.env` по инструкциям:

- [Первоначальная настройка](#)
- [Токены и секреты](#)

```
docker compose up -d
```

Проверить:

```
docker ps
```

Должны появиться:

- `symbolika-directus`
 - `symbolika-db`
-

4. Открыть новый Directus

В браузере:

```
http://IP_СЕРВЕРА:8057
```

Используйте `ADMIN_EMAIL` и `ADMIN_PASSWORD`, заданные в серверном `.env`. В пакете нет пароля по умолчанию.

5. Накатить структуру

Выполни:

```
docker exec -it symbolika-directus node /directus/setup/import-symbolika.mjs
```

Дождись сообщения:

```
Готово. Структура Символики создана.
```

6. Перезапустить Directus

```
docker restart symbolika-directus
```

7. Проверить коллекции

В новом Directus должны быть:

- Заказы
- Позиции заказа
- Технические задания позиций
- Платежи
- Распределение платежей
- Клиенты
- Компании
- Контрагенты
- Оплаты контрагентам
- Склад
- Категории склада

8. Важно поменять пароли

После проверки обязательно измени в `docker-compose.yml`:

- `ADMIN_PASSWORD`
- `POSTGRES_PASSWORD`
- `KEY`
- `SECRET`

Потом:

```
docker compose down
docker compose up -d
```

9. Старый Directus пока не удаляй

Сначала проверь новую систему на тестовом заказе.

Старые контейнеры:

- `directus-pg`
- `directus`
- `directus-db`

пока не трогай.

10. Когда всё проверишь

Можно будет переключить домен на новый порт/контейнер и только потом убрать старое.

Все инструкции системы «Символика»

Эта папка содержит пользовательские и эксплуатационные инструкции. Начните с нужного раздела.

PDF-версии

- [Скачать общий сборник «Все инструкции системы Символика»](#)
- [Оглавление в PDF](#)
- [Инструкция для менеджера в PDF](#)
- [Инструкция для производства и шелкографии в PDF](#)
- [Инструкция для дизайнера в PDF](#)
- [Инструкция для управляющего в PDF](#)
- [Инструкция для администратора в PDF](#)
- [Загрузка релиза на сервер в PDF](#)
- [Первоначальная настройка сервера в PDF](#)
- [Токены и секреты в PDF](#)
- [Настройка уведомлений сотрудников в PDF](#)
- [Настройка уведомлений клиентов в PDF](#)
- [Настройка встроенной почты в PDF](#)
- [Настройка Яндекс Диска в PDF](#)
- [Чек-лист релиза в PDF](#)
- [Чистая установка в PDF](#)
- [Исторический отчёт о релизе 10.08.2026 в PDF](#)
- [Актуальный отчёт о релизе 11.08.2026 в PDF](#)

Инструкции для сотрудников

- [Инструкция для менеджера](#)
- [Инструкция для производства и шелкографии](#)
- [Инструкция для дизайнера](#)
- [Инструкция для управляющего](#)
- [Полная инструкция для администратора](#)
- [Краткое оглавление ролевых инструкций](#)

Установка и выпуск

1. [Загрузка релиза на сервер](#)
2. [Первоначальная настройка сервера](#)
3. [Токены, пароли и переменные окружения](#)
4. [Чек-лист релиза](#)
5. [Чистая установка](#)
6. [Отчёт о готовности релиза 10.08.2026](#)

7. Актуальный отчёт о релизе 11.08.2026

Интеграции

- Настройка уведомлений сотрудников
- Настройка уведомлений клиентов
- Настройка встроенной почты
- Настройка Яндекс Диска

Рекомендуемый порядок первого запуска

1. Загрузить код и дампы по инструкции переноса.
2. Создать защищённый серверный `.env`.
3. Запустить базу и восстановить проверенный дампы.
4. Запустить Directus и применить актуальный SQL-слой.
5. Настроить домен и HTTPS.
6. Подключать интеграции по одной и проверять каждый канал.
7. Выполнить чек-лист релиза и приемочный сценарий.

Секретные значения нельзя записывать в эти файлы, `docker-compose.yml`, `.env.example` или Git. Они хранятся только в серверном `.env` с правами `600`.